

NNs for Implied Volatility surfaces





background

Option

Commonly used variable for option

S : the price of underlying asset

K : strike price, once the option holder execute the option, he can buy/sell the underlying asset at price K

τ : at time t , and the option expire at time T , then $T-t$ is τ , time to maturity

r : discount rate

q : dividend rate of the underlying asset

Option

Option is a often used financial derivatives, it gives buyers **the right**, but not obligation, **to buy or sell** the underlying asset **at a specific price** on **the expired date**

1. **Call**: gives the holder the right to **buy** the underlying asset at a predetermined price within a specified time frame.
2. **Put**: gives the holder the right to **sell** the underlying asset at a predetermined price within a specified time frame.

BS-formula

According to the **Black-scholes formula**, we know that the model assume a **normal distribution** on the probability distribution of the option price in calculating d1 and d2.

$$\begin{array}{ccccccc} \text{Call Price} & & \text{Risk-adj. prob.} & & \text{Risk-adj. prob.} & & \text{Discounts strike} \\ & & \text{that option won't} & & \text{that option will} & & \text{price to present} \\ & & \text{expire ITM} & & \text{expire ITM} & & \text{time} \\ \hline C(S, t) = & N(d_1) & S & - & N(d_2) & K & e^{-rT} \\ & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & & \underbrace{\hspace{1.5cm}} & \underbrace{\hspace{1.5cm}} & \\ & & \text{Stock Price} & & \text{Strike Price} & & \end{array}$$



[Black-scholes formula](#)

BS-formula

Under the risk-neutral assumption, there is Black-Scholes formula for option pricing, following is the formula :

$$c_{BSM} = Se^{(b-r)T} N(d_1) - Xe^{-rT} N(d_2) \quad (1.11)$$

$$p_{BSM} = Xe^{-rT} N(-d_2) - Se^{(b-r)T} N(-d_1), \quad (1.12)$$

where

$$d_1 = \frac{\ln(S/X) + (b + \sigma^2/2)T}{\sigma\sqrt{T}}$$

$$d_2 = d_1 - \sigma\sqrt{T}$$

$$b = r$$

gives the Black and Scholes (1973) stock option model.

$$b = r - q$$

gives the Merton (1973) stock option model with continuous dividend yield q .

σ : volatility of underlying asset

However, volatility cannot be directly acquired from market data!

BS-formula

And also by using Black-scholes formula and the real option price, we could calculate the Implied Volatility of the option, which is our expecting volatility of the price in the future.

$$\text{where } d_1 = \frac{\ln(S_0 / K) + (r - q + \sigma^2 / 2)T}{\sigma\sqrt{T}}$$

$$d_2 = \frac{\ln(S_0 / K) + (r - q - \sigma^2 / 2)T}{\sigma\sqrt{T}}$$

Motive

However, it is also the truth that some research indicate that the probability distribution is **not always follows** the normal distribution.

1. Fat tail
2. Leptokurtis

Other factors that would **influence the probability distribution**.

That is the reason why we have to use NN model to predict more precisely parameters in the model.

1. Data Preprocessing

Option data

First, we downloaded the data of Taiwan option (TXO) from Taiwan Futures Exchange Website, using the time period from 2009~2023.

The basic information we could get are :

Trading Date, Strike Price, Call/Put, Open, High, Low, Close, Volume, Settlement Price, Best Bid, Best Ask, Historical High, Historical Low, Increase or Decrease Percentage, Time to Maturity

Raw option data

Trading data for each option on each day encompasses a detailed record of all transactions involving that specific option contract during a particular trading day.

An option is uniquely identified by a combination of three attributes:

- Put/Call Indicator: (PUT or CALL)
- Expiration Date: (YYYY-MM-DD format)
- Strike Price: (Numerical value)

交易日期	契約	到期月份(週別)	履約價	買賣權	開盤價	最高價	最低價	收盤價	成交量	結算價	未沖銷契約數	最後最佳買價	最後最佳賣價	歷史最高價	歷史最低價	是否因訊息面暫停交易	交易時段	漲跌價	漲跌%
2009/01/05	TXO	200901	3600	買權	1140	1190	1060	1070	44	1100	252	1010	1160	1190	495		一般	80	8.08%
2009/01/05	TXO	200901	3600	賣權	3	4.5	2	4	3818	4	23879	3.6	4	349	2		一般	-1.9	-32.20%
2009/01/05	TXO	200901	3700	買權	1040	1040	1020	1030	3	1000	137	960	1000	1040	436		一般	140	15.73%

time to maturity



if issue year != expire year:

$$\text{tau} = \text{workdays covered in year1} / \text{total workdays in year1} + \text{workdays covered in year2} / \text{total workdays in year2}$$

(problem: cannot get workdays covered in the future)

(if total workdays in year2 is unknown: fill in 252)

else:

$$\text{tau} = \text{workdays covered} / \text{total workdays this year}$$

Concatenating discount and dividend rate

[In Deep smoothing of the implied volatility surface.\(2020\)](#), the authors proposed an method to calculate **implied interest rate** and **dividend** by using put-call parity.

This approach offers significant advantages:

1. It eliminates the need to search for historical data.
2. It represents the viewpoint of investors.

Result: see experiment1

Method to calculate implied interest rate and dividend rate

Collect data near the money (liquidity)

$$\text{call}(K, \tau) - \text{put}(K, \tau) = S e^{-q \tau} + K e^{-r \tau}$$

$$\text{call}(K, \tau) - \text{put}(K, \tau) = S \beta^S_\tau + K \beta^K_\tau$$

Using regression to find β^S_τ and β^K_τ

$$q_\tau = -\frac{\ln(\beta^S_\tau)}{\tau}$$

$$r_\tau = -\frac{\ln(\beta^K_\tau)}{\tau}$$

Put/Call Filter

Under a specific trading date, expiration date, and strike price, there are two options: one is a call and the other is a put. Theoretically, both options should exhibit the same implied volatility.

To optimize analysis, discard the option with the smaller volume and consider the one with higher liquidity. Remove extraneous columns from the original data and calculate τ using the expiration month (week).

Divide the data into two categories, put and call, and compare their volumes for further insights.

date	strike_price	put/call	volume	option_price	exdate	tau
2009-01-05	3600	買權	44	1100	2009-01-05	16
2009-01-05	3700	買權	3	1000	2009-01-05	16
date	strike_price	put/call	volume	option_price	exdate	tau
2009-01-05	3600	賣權	3818	4	2009-01-05	16

model design

model input & output

input:

log moneyness($m = \ln(S_t/K)$),

time to maturity(τ)

output:

total variance ω : defined as $\sigma^2 \tau$

Other important variable:

total variance data when AT THE MONEY:

$\omega_{atm}(\tau)$

model: $R \mid R^2 \rightarrow R, \omega(m, \tau)$

total variance: $\omega(m, \tau)$

At the money: when $K = S$

$\omega_{atm} : \omega_{atm}(\tau)$

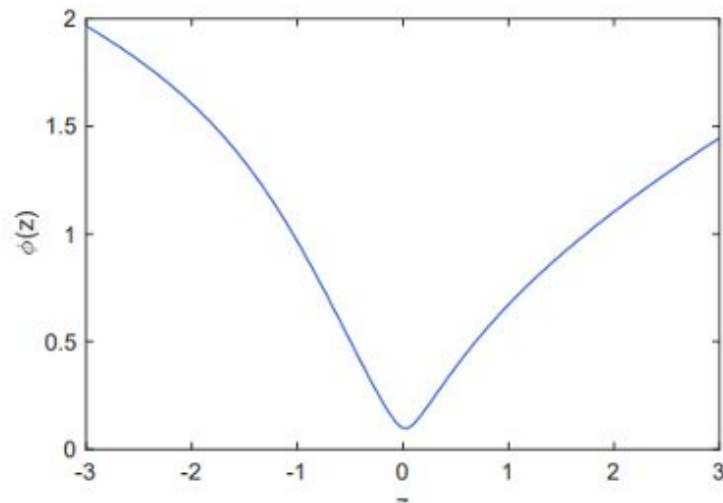
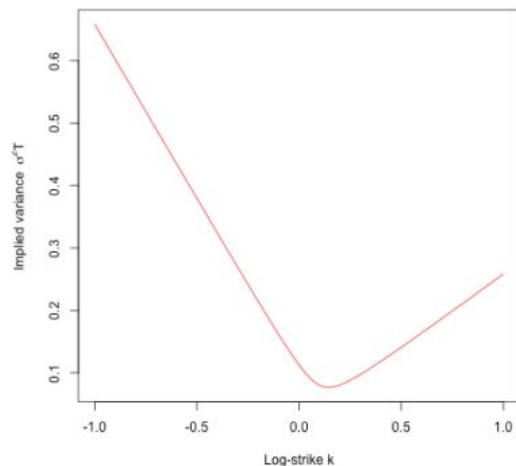
explainable model vs blackbox

Nowadays, neural network(NN) has been widely used in finance field, however, there is some question that NN is a blackbox.

During the fitting process, we don't know the relation of input and output.

So we try to integrate financial theory into NN structure

explainable model



DEFINITION 1 (Smile Function). For any $z \in \mathbb{R}$,

$$\phi(z) = \left[z \tanh\left(z + \frac{1}{2}\right) + \tanh\left(-\frac{1}{2}z + \epsilon\right) \right]^{1/2}, \quad z \in \mathbb{R},$$

Because implied volatility to log moneyness ($k = \ln(K/S)$) curve is similar to one of the smile function, if we input log moneyness to smile function ($\phi(k)$), we can fit the implied volatility better

prior model-SSVI

from: Arbitrage-free SVI volatility surfaces Jim Gatheral* , Antoine Jacquier† March 22, 2013

$$\omega_{\text{prior}}^{\text{ssvi}}(k, \tau) = \frac{w_{\text{atm}}(\tau)}{2} \left(1 + \rho \phi(w_{\text{atm}}(\tau))k + \sqrt{(\phi(w_{\text{atm}}(\tau))k + \rho)^2 + 1 - \rho^2} \right)$$
$$\phi(x) = \frac{\eta}{x^\gamma(1+x)^{1-\gamma}}$$

rho, eta, gamma: trainable variable

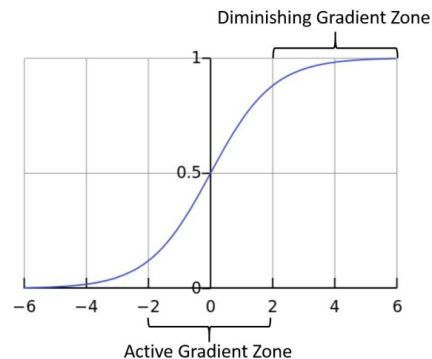
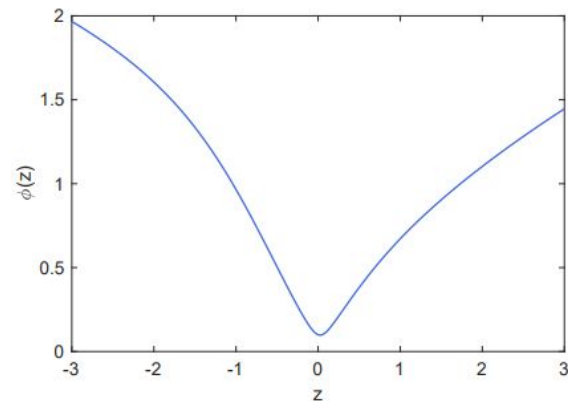
prior model-smile

J : number of neurons in input layer

$$y_i = \sum_{j=1}^J \phi(m\bar{w}_j^{(i)} + \bar{b}_j^{(i)}) \psi(\tau\tilde{w}_j^{(i)} + \tilde{b}_j^{(i)}) e^{\hat{w}_j^{(i)}} + e^{\hat{b}^{(i)}},$$

$$\phi(z) = \left[z \tanh\left(z + \frac{1}{2}\right) + \tanh\left(-\frac{1}{2}z + \epsilon\right) \right]^{1/2}, \quad z \in \mathbb{R}, \quad A = \frac{1}{1+e^{-x}}$$

$$\Psi(z) = \text{sigmoid}(z)$$



ensemble learning

I : number of single model

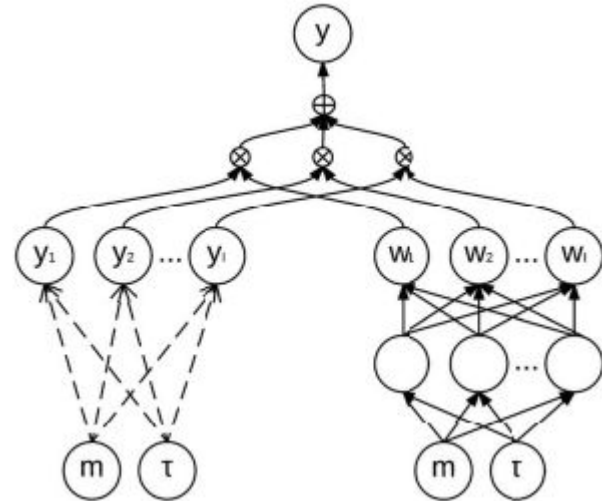
K : number of neurons in weight model

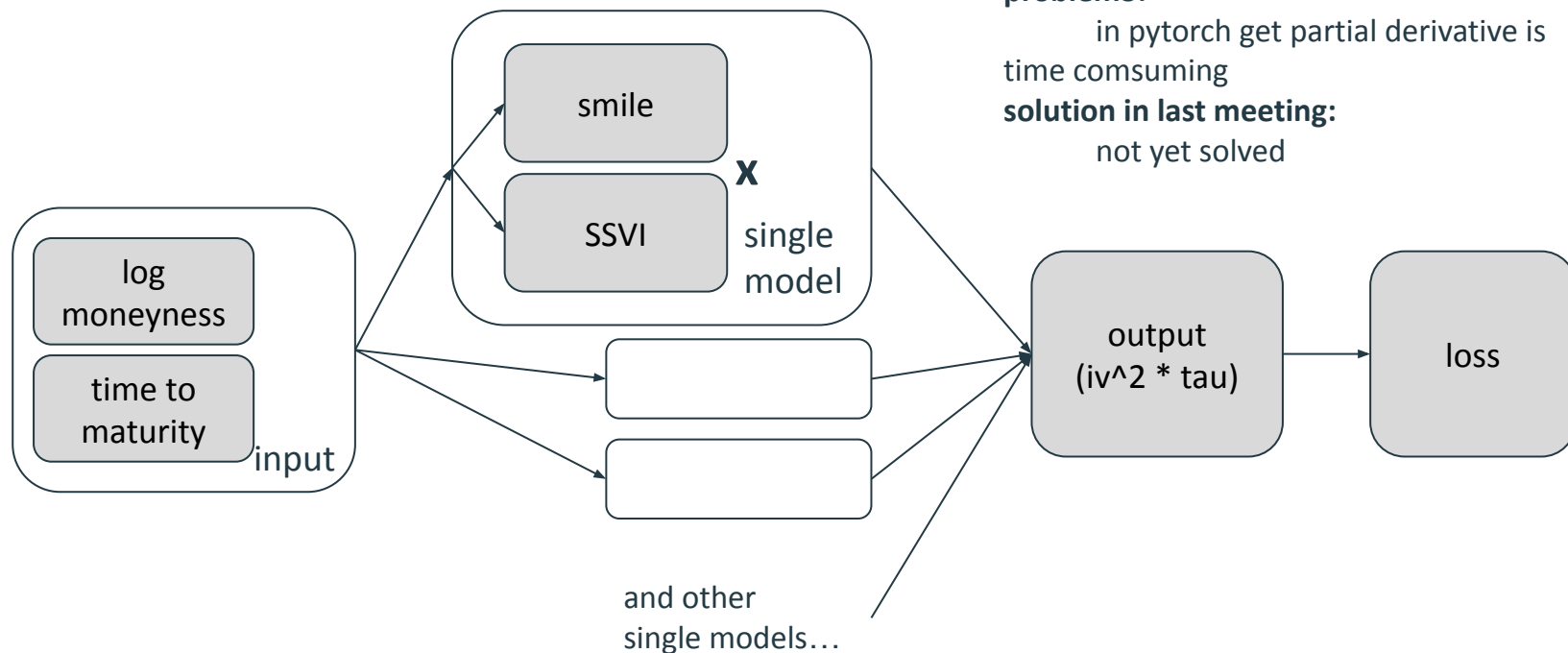
$$y' = y_{ssvi}' * y_{smile} + y_{smile}' * y_{ssvi}$$

$$y_i = y_{ssvi} \times y_{smile}$$

$$w_i(m, \tau) = \frac{e^{\sum_{k=1}^K \sigma_2(m\dot{W}_{1,k} + \tau\dot{W}_{2,k} + \dot{b}_k) \ddot{W}_{k,i} + \ddot{b}_i}}{\sum_{i=1}^I e^{\sum_{k=1}^K \sigma_2(m\dot{W}_{1,k} + \tau\dot{W}_{2,k} + \dot{b}_k) \ddot{W}_{k,i} + \ddot{b}_i}}$$

$$y(m, \tau) = \sum_{i=1}^I y_i(m, \tau) w_i(m, \tau).$$





problem3:

in pytorch get partial derivative is time consuming

solution in last meeting:

not yet solved

problem2:

Deep Smoothing of the Implied Volatility Surface : pred_target: total variance($iv^2 * \tau$)

Incorporating Prior Financial Domain Knowledge... : pred_target: implied volatility(iv)

solution in last meeting:

按照problem1內solution, 統一模型pred_target

Loss function

prediction error-RMSE & MAPE

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

$\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n$ are predicted values

$y_1, y_2, \dots, y_n$ are observed values

n is the number of observations

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{actual_i - predicted_i}{actual_i} \right|$$

n = number of considered points

$actual_i$ = actual value

$predicted_i$ = the predicted value

Total prediction error = RMSE + MAPE



experiment

experiment1

fitting implied interest rate and dividend rate using put call parity

Fail: Assume that interest and dividend rate are different each day, group data on each date and time to maturity

Result: too much irrational result, ex: e^{-rt} or $e^{-qt} > 1$

```
date,tau,0
2009-01-05,0.06349206349206349,[ 0.99180399 -0.99928571]
2009-01-05,0.1746031746031746,[ 1.03364289 -1.04642857]
2009-01-05,0.2857142857142857,[ 1.04360852 -1.05821429]
2009-01-05,0.6468253968253969,[ 1.01089907 -1.03      ]
2009-01-05,1.007936507936508,[ 0.84047547 -0.889      ]
2009-01-06,0.05952380952380952,[ 0.98800054 -0.99267857]
2009-01-06,0.17063492063492064,[ 1.02228133 -1.03428571]
2009-01-06,0.28174603174603174,[ 1.07460722 -1.08785714]
2009-01-06,0.6428571428571429,[ 1.03096051 -1.0455      ]
2009-01-06,1.003968253968254,[ 0.93339834 -0.977      ]
2009-01-07,0.05555555555555555,[ 0.98166492 -0.98892857]
2009-01-07,0.16666666666666666,[ 0.97203139 -0.98714286]
2009-01-07,0.2777777777777778,[ 1.05297685 -1.06857143]
2009-01-07,0.6388888888888888,[ 0.94721687 -0.965      ]
2009-01-07,1.0,[ 0.9069231 -0.955      ]
2009-01-08,0.051587301587301584,[ 0.9901683 -1.0075      ]
2009-01-08,0.1626984126984127,[ 0.99636507 -1.01714286]
2009-01-08,0.27380952380952384,[ 0.96370407 -0.98714286]
2009-01-08,0.6349206349206349,[ 0.97668028 -1.0045      ]
2009-01-08,0.996031746031746,[ 0.79027324 -0.844      ]
2009-01-09,0.047619047619047616,[ 0.9713605 -0.98910714]
2009-01-09,0.15873015873015872,[ 1.0181536 -1.03892857]
2009-01-09,0.2698412698412698,[ 1.12980825 -1.14785714]
2009-01-09,0.6309523809523809,[ 1.18783662 -1.211      ]
2009-01-09,0.9920634920634921,[ 0.77319869 -0.833      ]
```

experiment2

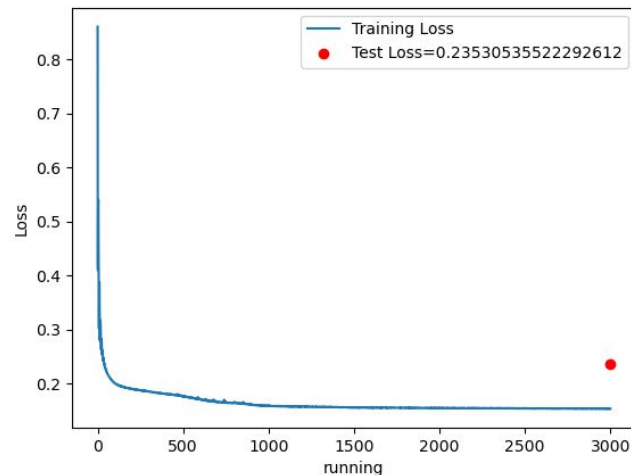
using given dataset to test model performance

Fail:

- 1.huge difference between training loss and testing loss
- 2.obvious prediction incorrect

Possible reason:

- 1.Too much noise for TAIEX data
- 2.Model overfitting



	impl_volatility	impli_vol_pred
64554	0.167506	0.151277
64555	0.167621	0.138677
64556	0.166311	0.132479
64557	0.163206	0.127032
64558	0.159497	0.123090

problem

- 1.改善速度過慢問題可能原因:取得變數偏微分的過程
- 2.更改模型構造, 解決無法收斂、loss爆炸問題, incorporate x SSVI -> NN x incorporate

problem 1

1. old method: autograd

```
output = self.output_layers(out_hidden)

total_output = torch.sum(output)
grad1 = torch.autograd.grad(total_output, (ttm, logm), retain_graph=True, create_graph=True)
grad_ttm1 = grad1[0].clone()
grad_logm1 = grad1[1].clone()
total_grad_logm1 = torch.sum(grad_logm1)
grad_logm2 = torch.autograd.grad(total_grad_logm1, logm, retain_graph=True)[0]
return output, grad_ttm1, grad_logm1, grad_logm2
```

2. new method: Jacobian & Hessian matrix

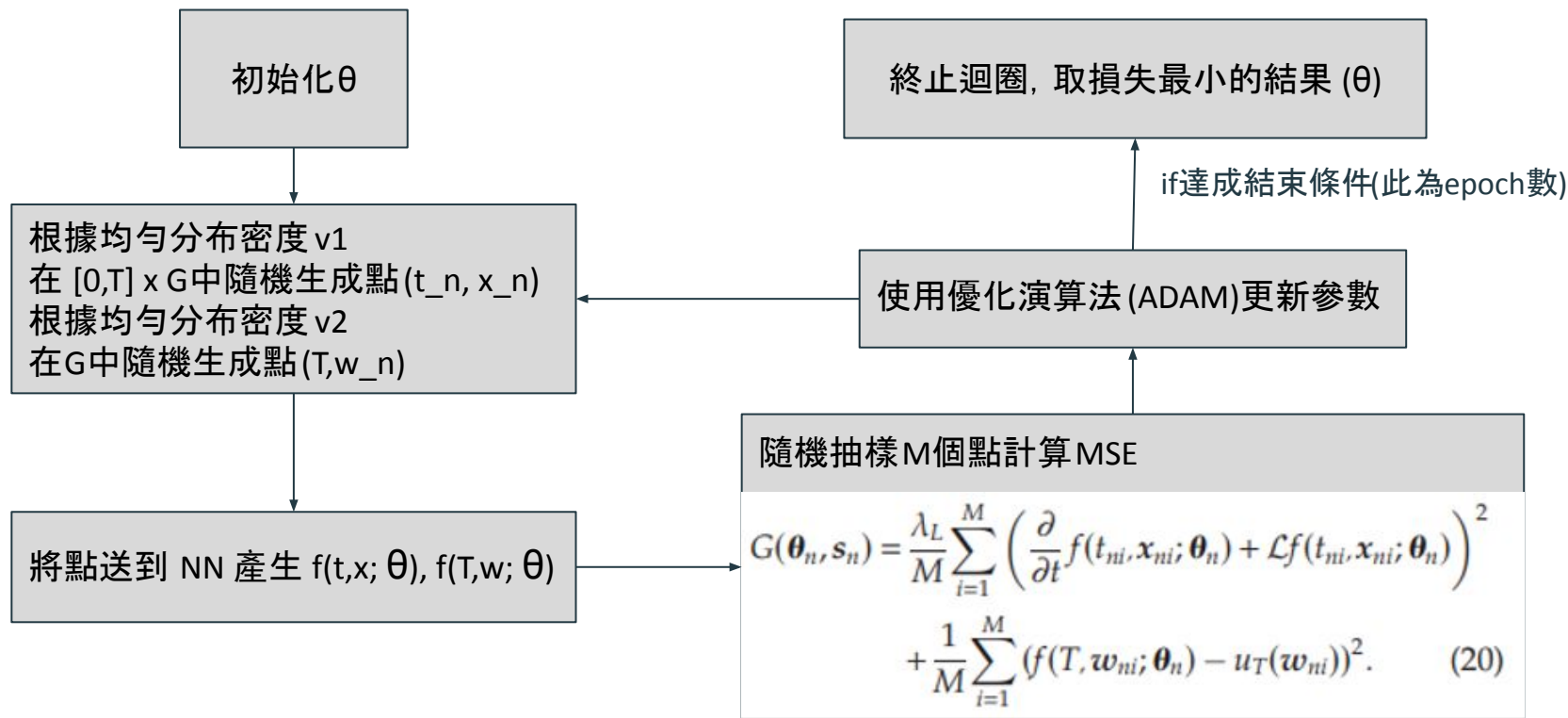
`torch.autograd.functional.jacobian()`

`torch.autograd.functional.hessian()`

2. new method: 手動

problem 2

$$\omega_{nn}(k, \tau; \theta_1) = \bigcirc_{i=1}^{n+1} f_i^{W_i, b_i}(k, \tau) \text{ with } f_i(x) = \begin{cases} g_i(W_i x + b_i) & i < n+1 \\ \alpha(1 + \tanh(W_{n+1} x + b_{n+1})) & i = n+1 \end{cases}$$



由於 θ 可能只收斂到區域最小值而非全域最小值, 因此需要調整[hyperparameter](#)

λ_L

NN DL學習方法的實踐(GBM)

幾何布朗運動其CDF的backward Kolmogorov equation如下：

$$\frac{\partial C}{\partial t} - \frac{\sigma^2}{2} \frac{\partial^2 C}{\partial x^2} = 0, t \in [0, T), x \in \mathbb{R}, \quad (22)$$

$$C(T, x) = \mathbf{1}(x \leq y) = \begin{cases} 1, & x \leq y, \\ 0, & x > y. \end{cases} \quad (23)$$

從資產價格過程中我們可以知道CDF是波動率 σ 的函數。

因此該問題需要在 (t, x, y, σ) domain $Q \in \mathbb{R}^4$ 中求解

該domain不僅包含自變數 $(t, x) \in [0, T] \times G \in \mathbb{R}^2$ 的公式還包含參數 $(y, \sigma) \in \mathbb{R}^2$

如前一章描述的使用 NN $f(x; \theta) = f(t, x, y, \sigma; \theta)$

我們可以為backward Kolmogorov equation找到這個參數問題的解

$$L_1(f) = \left\| \left(\frac{\partial}{\partial t} - \frac{1}{2} \sigma^2 \frac{\partial^2}{\partial x^2} + \frac{1}{2} \sigma^2 \frac{\partial^2}{\partial x^2} \right) f(t, x, y, \sigma; \theta) \right\|_{[0, T] \times G, v_1}^2, \quad (24)$$

$$L_2(f) = \|f(T, x, y, \sigma; \theta) - \mathbf{1}(x \leq y)\|_{G, v_2}^2, \quad (25)$$

$$L(f) = \lambda_L L_1(f) + L_2(f). \quad (26)$$

domain $Q : x, y \in [-2.3, 2.3], \sigma \in [0, 0.6], t \in [0, 1.2]$

我們注意到，為 x 和 y 的範圍選擇 ± 2.3 能大約對應到 $\pm 3.5 \sigma_{\max} t_{\max}$ 。

考量的時間範圍為 $T=1.2$ 。

($T-t$: time to maturity, 未來會用 T 表示，但這不該引起混淆)

一旦 NN 離線訓練完成，就可在線用於具有此基礎模型的任何期權定價(有訓練到的 σ, T 範圍內)

儘管對數現貨價格 $x_0 = 0$ ，但我們仍然需要在 x 中訓練網路，因為 PDE 是用 x 寫入的。

本文中使用的 DGM 網路可以被視為長短期記憶網路（LSTM）的變體。

LSTM 是種特殊的RNN，可用於對序列資料進行建模，如時間序列或語音辨識。

LSTM 型架構適用於在初始或終端條件下出現“急轉彎”的 PDE 問題，

這正是我們在此遇到的，因為在我們估計的反向 Kolmogorov 方程的終端條件下有一個階躍函數

$$S^1 = \vartheta(W^1 x + b)$$

$$Z^\ell = \vartheta(U^{z,\ell} x + W^{z,\ell} S^\ell + b^{z,\ell}), \ell = 1, \dots, L,$$

$$G^\ell = \vartheta(U^{g,\ell} x + W^{g,\ell} S^\ell + b^{g,\ell}), \ell = 1, \dots, L,$$

門閥 $R^\ell = \vartheta(U^{r,\ell} x + W^{r,\ell} S^\ell + b^{r,\ell}), \ell = 1, \dots, L,$

$$H^\ell = \vartheta(U^{h,\ell} x + W^{h,\ell} (S^\ell \odot R^\ell) + b^{h,\ell}), \ell = 1, \dots, L,$$

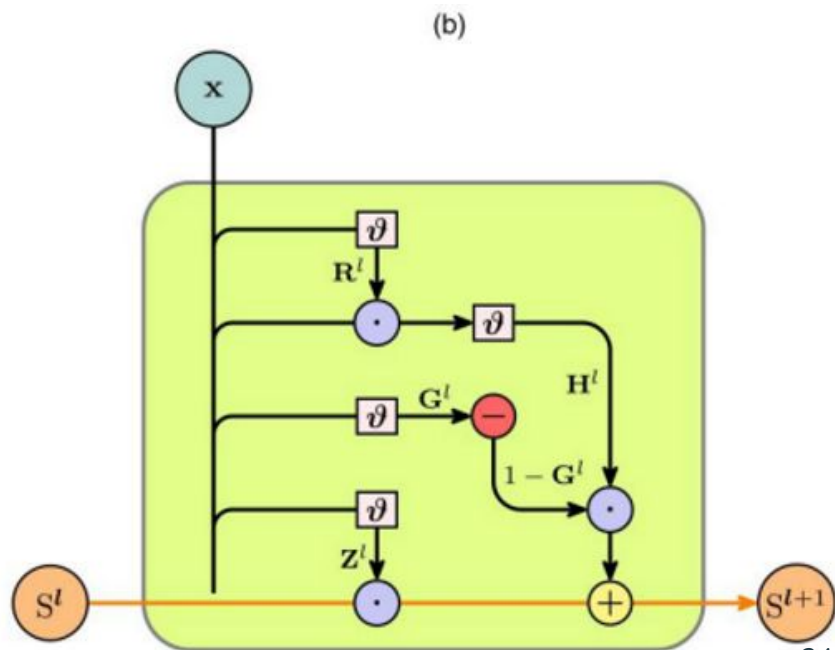
$$S^{\ell+1} = (1 - G^\ell) \odot H^\ell + Z^\ell \odot S^\ell, \ell = 1, \dots, L,$$

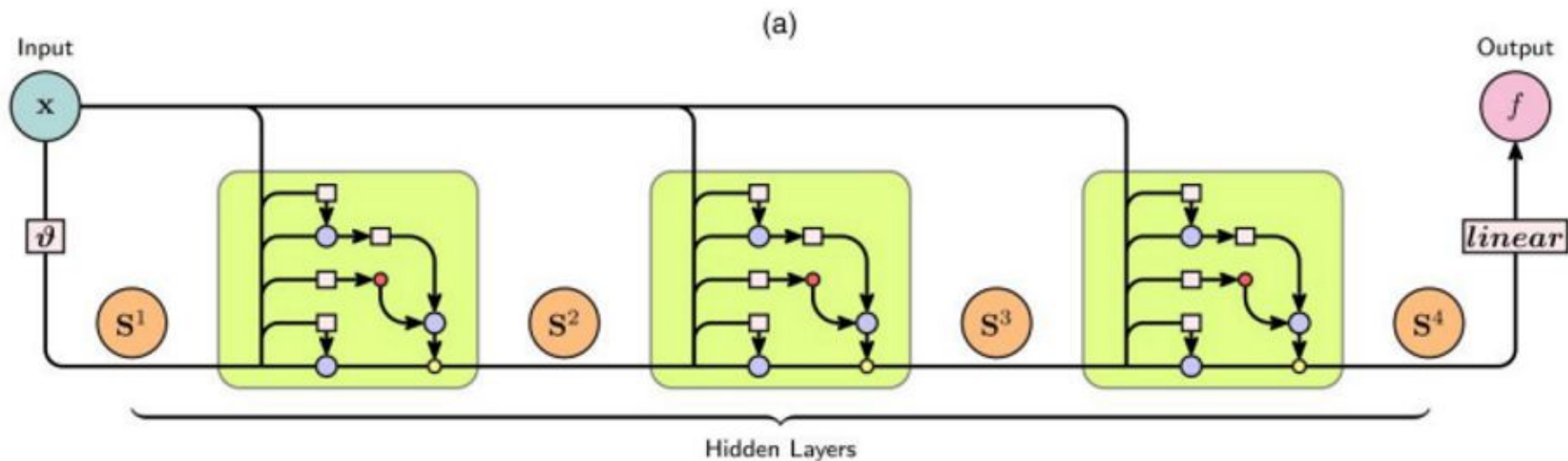
$$f(x; \theta) = WS^{L+1} + b,$$

x : input($D \times 1$, : D 可訓練特徵的數量)

$\vartheta(\cdot)$: 激勵函數, $\odot$: 點乘

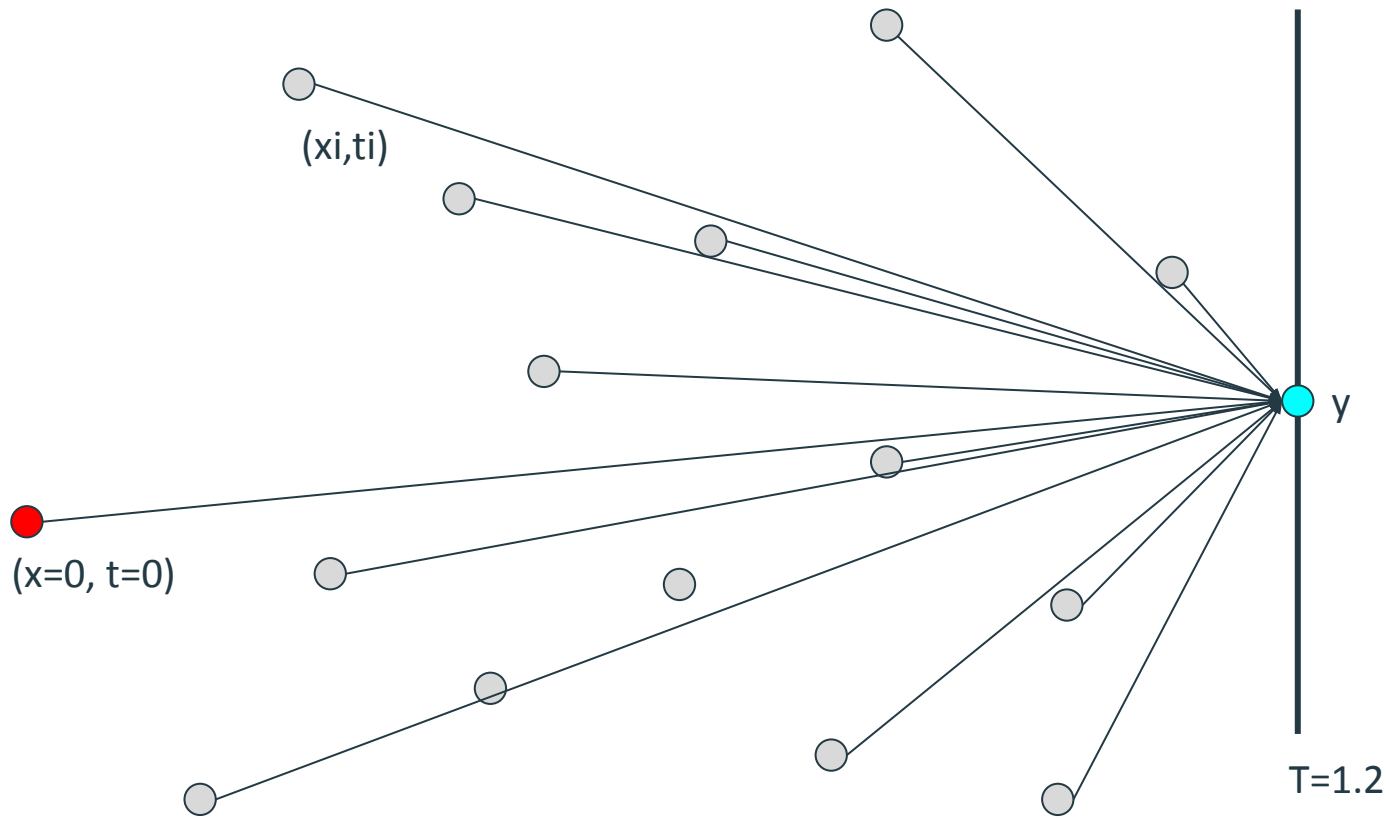
U, W, b : NN參數, 用來形成 $\circ$ ($U, W : M \times D$; $b : M \times 1$)





隱藏層有 $L+1$ 層(1層初始隱藏層 + L 層LSTM)

輸入 x 可以是來自 $[0, T] \times G$ 的 (t, x) ，用於求解具有固定參數的 PDE 問題，或者當我們求解參數化 PDE 問題時，也可以包括 PDE 的參數。



λL

訓練精度通常隨 epoch (或採樣點) 而增加。

一定量的訓練后，RMSE 的下降速度趨緩：

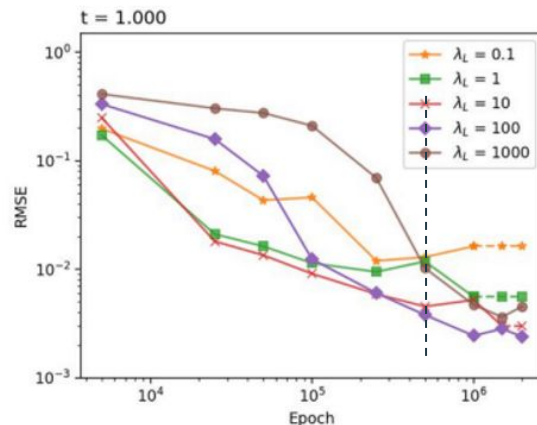
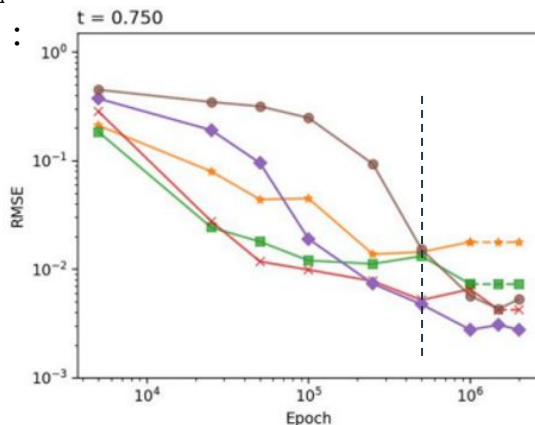
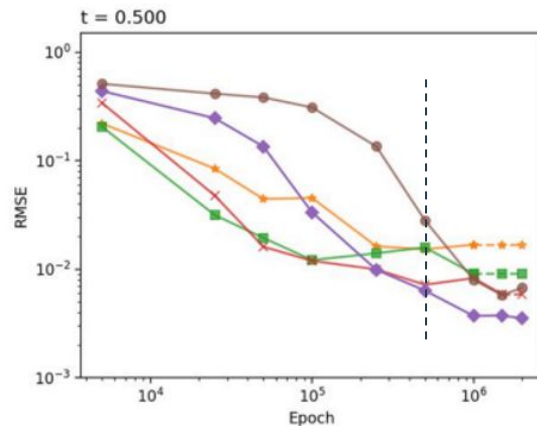
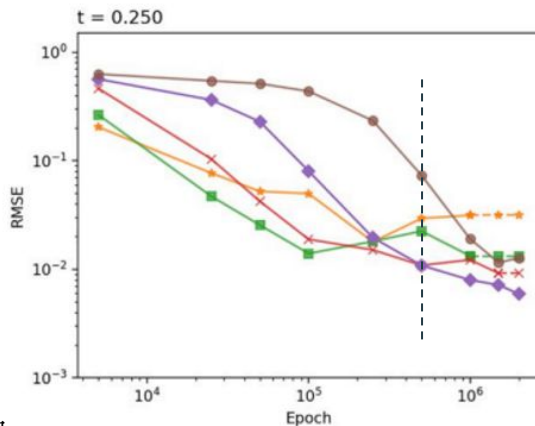
$\lambda L = 1$ or 10 時尤其明顯

這是因為DL中的梯度消失問題

$\lambda L = 10$ 時，梯度消失問題得到緩解；

$\lambda L = 100$ 有助於進一步提高性能；

$\lambda L = 1000$ 時性能變得更差。



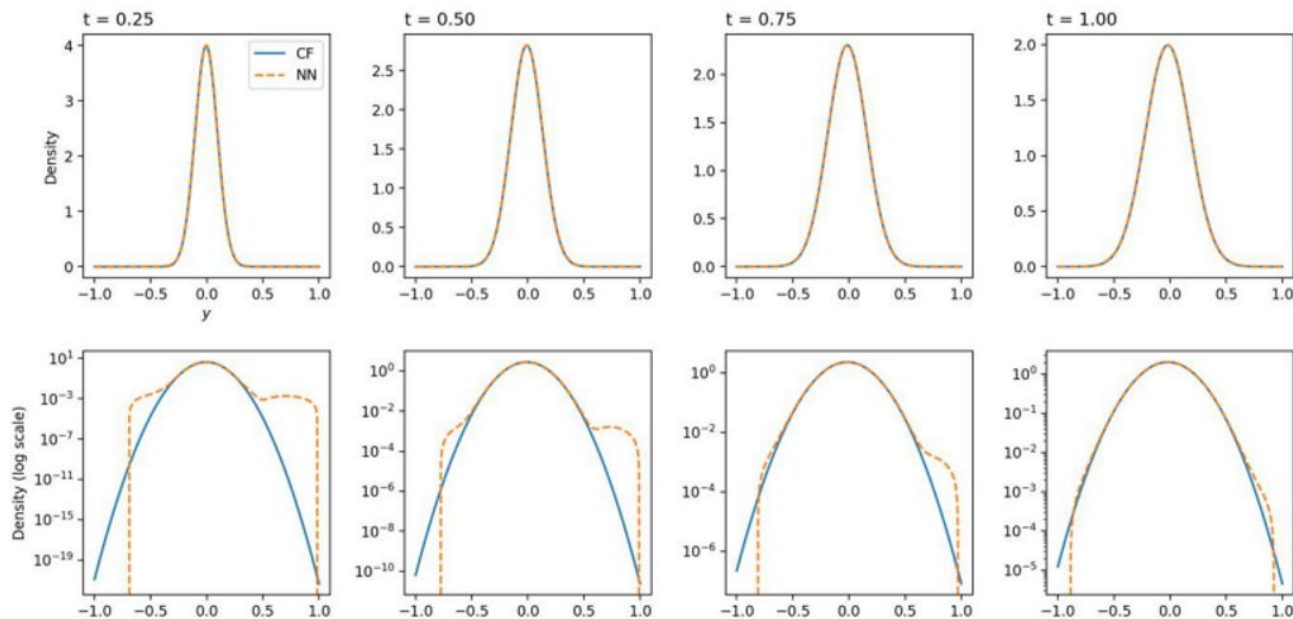
NN 近似值可以非常接近 RMSE 的精確 TPDF。

然而考慮相對近似時，估計在中心附近準確，尾部則不準確。

具體來說對於小於 10^{-3} 的密度值，近似值無法產生高度準確的結果。

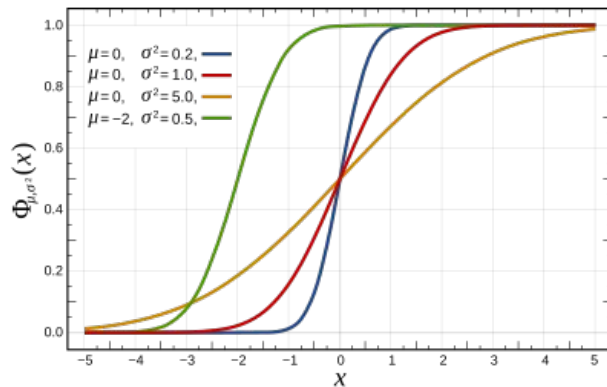
因訓練旨在最小化 L2 損失，即均方誤差。

並且尾部會產生大百分比的誤差，這是我們方法的局限性。



(maturity = 1.2)

一般情況 ($\tau \geq 0.1$) $t \in [0, 1.2]$	接近到期 ($\tau < 0.1$) $t \in [1.08, 1.2]$
$\lambda L = 100$ 效果較好	$\lambda L = 1$ 效果較好
x 範圍大效果較好 [-2.3, 2.3]	x 範圍小效果較好 [-0.75, 0.75]



Transfer learning 可解決不同 τ 的 NN 訓練所帶來的額外複雜性。

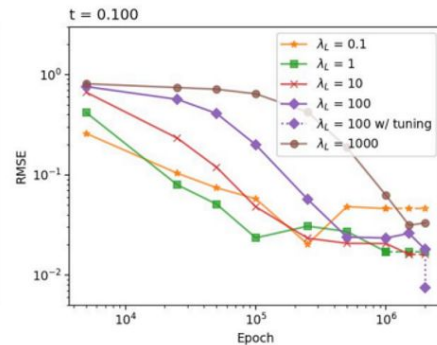
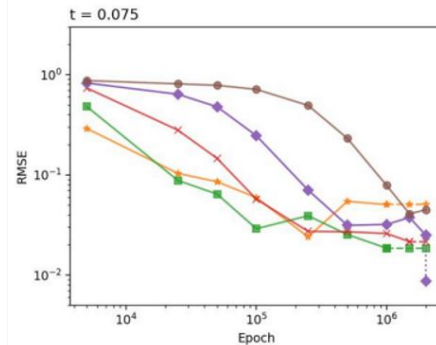
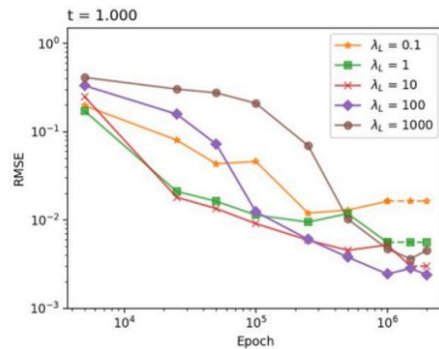
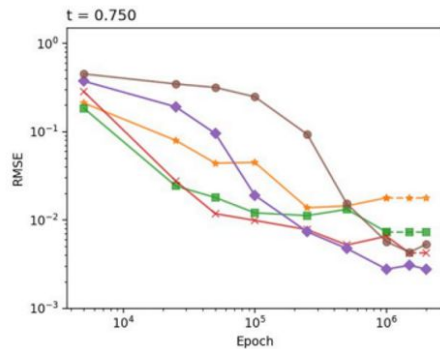
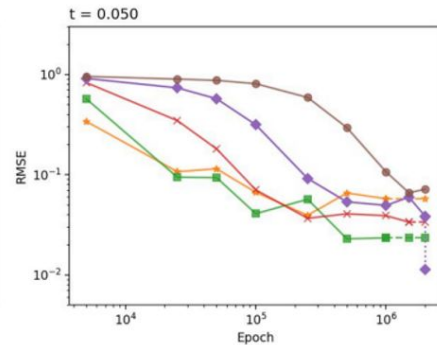
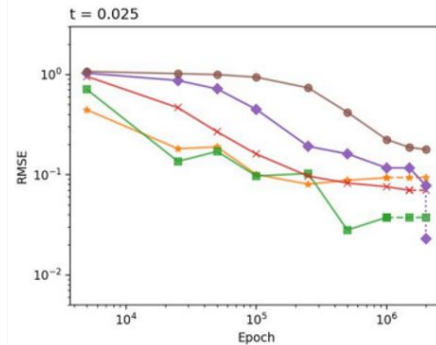
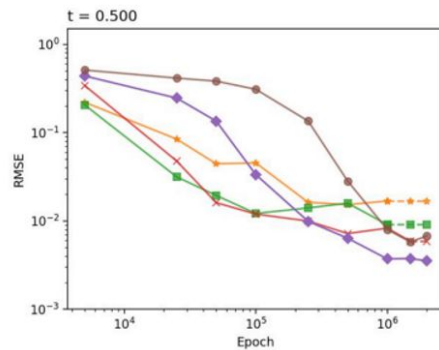
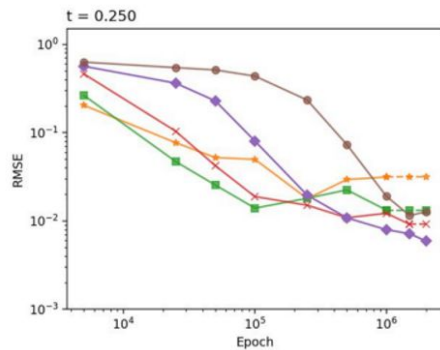
Transfer learning: 用已針對相關問題訓練的 NN 的知識, 來訓練新的 NN 來解決問題。

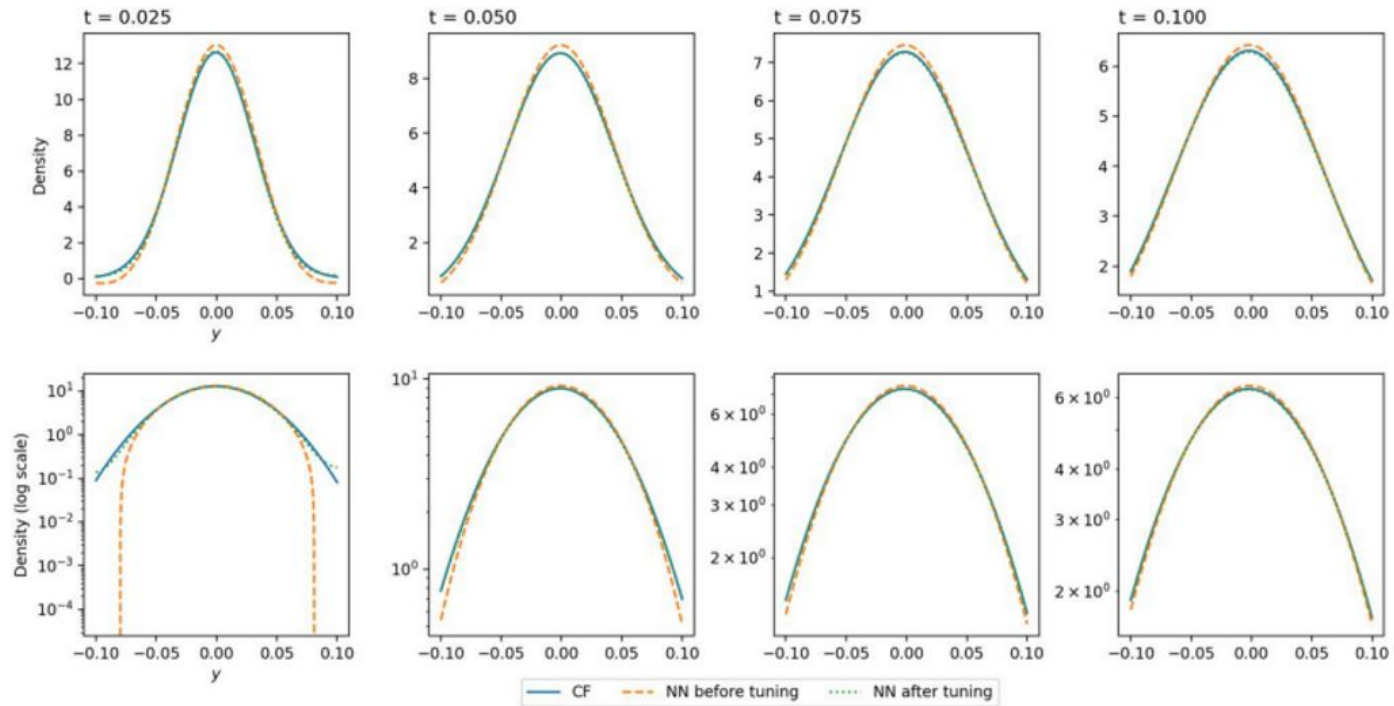
此處: 依原方法、一般情況下的超參數訓練完後, 用已訓練的 NN 初始化新模型

換新的超參數後只使用另外 2,500 個 epoch 訓練一個 NN, 並選擇總損失最小的 NN。

我們只使用少量的 epoch 來微調新域的 NN, 因已 $\lambda L = 100$ 更改為 1

使用更多 epoch 進行訓練會使失去 $\lambda L = 100$ 訓練時帶來的整體準確性





在本節中，我們通過期權定價測試神經 TPDF 的準確性。

表 1: GBM 流程下，一組給定參數的put的定價表現。

所有絕對誤差都小於 0.0005相當準確

但是deep OTM的相對誤差很大

因為deep OTM put的價格非常接近0。

此外對當價格極小時，可能會給出負價格。

Strikes	Price		Error	
	CF	NN	Abs ($\times 10^{-4}$)	%
0.5	9.4310×10^{-6}	-2.524×10^{-4}	2.62	2,776.14%
0.6	2.6111×10^{-4}	-6.865×10^{-5}	3.30	126.29%
0.7	2.4811×10^{-3}	2.1189×10^{-3}	3.62	14.60%
0.8	0.0119	0.0115	3.73	3.14%
0.9	0.0359	0.0355	3.91	1.09%
1.0	0.0797	0.0792	4.26	0.53%
1.1	0.1429	0.1425	4.61	0.32%
1.2	0.2215	0.2210	4.89	0.22%
1.3	0.3101	0.3096	4.97	0.16%
1.4	0.4045	0.4040	4.78	0.11%
1.5	0.5019	0.5015	4.37	0.09%
Average time	0.002	0.245		

Maturity	Error type	DOTM	OTM	ATM	ITM	DITM
$t = 0.25$	PrPCTE	5.662445×10^{12}	2.933544	0.006059	0.003086	0.001537
	PrRMSE	0.000265	0.000204	0.000331	0.000463	0.000539
$t = 0.5$	PrPCTE	1.448508×10^5	0.083452	0.003751	0.002292	0.001373
	PrRMSE	0.000248	0.000218	0.000279	0.000409	0.000518
$t = 0.75$	PrPCTE	3.338457×10^2	0.025648	0.002880	0.001746	0.001089
	PrRMSE	0.000237	0.000225	0.000264	0.000369	0.000470
$t = 1.0$	PrPCTE	14.51919	0.013960	0.002432	0.001437	0.000899
	PrRMSE	0.000232	0.000228	0.000253	0.000337	0.000429

單個標的 $S(s)$ 遵循在forward measure下的隨機微分方程 (SDE)

$$dS = \sigma(s, S)SdW, S(t) = S_0. \quad (1)$$

$W(s)$ ：是一維標準維納過程， s ：某時間點
波動率 $\sigma(s, x)$ 是確定性函數(同input則必同output)。

若忽略折現因素

European option price可表示為：

$$V(0, S_0) = E[f(S_{0, S_0}(T))].$$

$S_{0, S_0}(s)$ ：時間點0、初始股價為 S_0 條件下，標的在時間點 s 的價格

因為標的SDE(1)是Markov process, 所以也可寫成：

$$V(0, S_0) = \int_0^{+\infty} f(y)p(0, S_0; T, y)dy,$$

$p(t, S_0; T, y)$ ：隨機變數 $S_{T, S_0}(T)$ 的轉移機率密度函數 (TPDF)
時間點 t 、股價為 S_0 的條件下，股價在時間點 T 時為 y 的機率

為了拉伸phase space, 決定採用對數資產價格空間
則變化後的選擇權方程式為：

$$V(0, S_0) = \int_{-\infty}^{+\infty} f(e^y)p(0, x_0; T, y)dy,$$

$p(t, x; T, y)$ ：隨機變數 $X_{t,x}(T)$ 的 TPDF
時間點 t 、對數股價為 x 的條件下，對數股價在時間點 T 時為 y 的機率

當波動率 σ 與時間無關時，不失一般性
我們可以將 $X(s)$ 在時間 $t=0$ 的機率
用更簡單的符號 $p(x; s, y)$ 或 $p(s, y)$ 而不是 $p(0, x; s, y)$ 表示。

目標：估計對數價格的TPDF

Fokker-Planck(forward Kolmogorov)方程
控制了SDEs解的轉移概率的時間演化。
在對數價格的情況下，它採用以下形式：

$$\frac{\partial}{\partial s} p(s, y) = \frac{1}{2} \frac{\partial^2}{\partial y^2} [\sigma^2(s, e^y) p(s, y)] + \frac{1}{2} \frac{\partial}{\partial y} [\sigma^2(s, e^y) p(s, y)], \quad s > 0, y \in \mathbb{R}, \quad (7)$$

$$p(s, y) = p(0, x; s, y)$$

具有x固定時的初始條件，

$$p(0, y) = \delta(y - x), \quad y \in \mathbb{R}, \quad (8)$$

$\delta(\cdot)$: the Dirac delta function。

$$\delta(y - x) = \begin{cases} +\infty, & y = x, \\ 0, & y \neq x, \end{cases} \quad (9)$$

且 $\int_{-\infty}^{+\infty} \delta(y - x) dy = 1$ 。

因函數在點 $y=x$ 處無限，故很難用數值方式求解(7)-(8)。

$$C(t, x; T, y) := \text{Prob}(X_{t,x}(T) \leq y) = \int_{-\infty}^y p(t, x; T, z) dz,$$

在對數價格的情況下，累積分布滿足backward Kolmogorov 方程：

$$\frac{\partial}{\partial t} C(t, x) + \frac{1}{2} \sigma^2(t, e^x) \frac{\partial^2}{\partial x^2} C(t, x) - \frac{1}{2} \sigma^2(t, e^x) \frac{\partial}{\partial x} C(t, x) = 0, \quad t \in [0, T), x \in \mathbb{R}, \quad (11)$$

$$C(t, x) = C(t, x; T, y)$$

具有y固定時的終端條件，

$$C(T, x) = \mathbf{1}(x \leq y) = \begin{cases} 1, & x \leq y, \\ 0, & x > y, \end{cases} \quad x \in \mathbb{R}, \quad (12)$$

其中 $C(T, x) = C(T, x; T, y)$

$$p(t, x; T, y) = \frac{\partial}{\partial y} C(t, x; T, y).$$

累積機率分布 (CDF)

$$C(t, x; T, y) := \text{Prob}(X_{t,x}(T) \leq y) = \int_{-\infty}^y p(t, x; T, z) dz,$$

$$\begin{aligned} \frac{\partial}{\partial t} C(t, x) + \frac{1}{2} \sigma^2(t, e^x) \frac{\partial^2}{\partial x^2} C(t, x) \\ - \frac{1}{2} \sigma^2(t, e^x) \frac{\partial}{\partial x} C(t, x) = 0, t \in [0, T), x \in \mathbb{R}, \end{aligned} \quad (11)$$

如固定 y (選擇權履約價固定)

則終端($t=T$)條件如下:

$$C(T, x) = \mathbf{1}(x \leq y) = \begin{cases} 1, & x \leq y, \\ 0, & x > y, \end{cases} x \in \mathbb{R}, \quad (12)$$

(11)-(12)屬於(14)-(15)的形式

Wiener過程驅動的任何 SDE的backward Kolmogorov都具有這種形式。

拋物線偏微分方程

$$\frac{\partial}{\partial t} u(t, x) + \mathcal{L}u(t, x) = 0, (t, x) \in [0, T) \times G, \quad (14)$$

具有終端條件:

$$u(T, x) = u_T(x), x \in G, \quad (15)$$

和邊界條件:

$$u(t, x) = g(t, x), (t, x) \in [0, T] \times \partial G.$$

目標:

訓練一個 $f(t, x; \theta)$ 來近似 $u(t, x)$

θ : 可訓練參數

Loss function

$$L_1(f) = \left\| \frac{\partial}{\partial t} f(t, x; \theta) + \mathcal{L}f(t, x; \theta) \right\|_{[0, T] \times G, v_1}^2, \quad (17) \longrightarrow \frac{\partial}{\partial t} u(t, x) + \mathcal{L}u(t, x) = 0, (t, x) \in [0, T] \times G, \quad (14)$$

$$L_2(f) = \|f(T, x; \theta) - u_T(x)\|_{G, v_2}^2, \quad (18) \longrightarrow u(T, x) = u_T(x), x \in G, \quad (15)$$

$$L(f) = \lambda_L L_1(f) + L_2(f), \quad (19)$$

加權項

我們選擇 $v_1(y)$ 表示機率密度在時間-價格域 $[0, T] \times G$ 上均勻分佈

我們選擇 $v_2(y)$ 表示機率密度在價格域 G 上均勻分佈

positive probability density on a domain $\mathcal{Y}$. In the context of this paper's application, we choose $v_1(y)$ as the probability density for uniform distribution on the time-price domain $\mathcal{Y} = [0, T] \times G$ and $v_2(y)$ as the probability density for uniform distribution on the price domain $\mathcal{Y} = G$.

具體任務：

訓練 θ ，以近似 $u(t, x)$ 、最小化 $L(f)$

$$\text{低估率} = \frac{\text{當日價格被低估的選擇權種類數量}}{\text{當日所有交易的選擇權種類數量}}$$

VIX 指數變化量：今天的 VIX 指數 — 昨天的 VIX 指數

在大事件發生日時仍有過高的問題，是否為不滿足4.5式所導致

採用其中預測效果最優的模型(CV + Cyclic learning Rate + With Syn data)，選擇 2016/6作為基準月

3 個大事件發生日作為觀察日，分別為

- 2001/9/17(7,,,10,,,,,,17)，美國 911 事件後的第一個開盤日
- 2008/10/15，次貸危機股價波動最高日
- 2020/3/16，新冠疫情股價波動最高日

日期	2016/06 平均	2001/09/17	2008/10/15	2020/03/16
CV Model 低估率	59.86%	86.52%	87.14%	99.93%
Multi Model 低估率	52.53%	97.14%	91.31%	99.45%

嘗試導入變數以改善模型於 重大事件日期效能之實驗

1. 研究背景

- 許珈瑋學姊的 CV model 為選擇權定價提供了良好的預測精準度
- 但是在發生重大事件的日子, 存在 MSE(mean square error)過高的問題
- 在游竣瑜學長的研究中發現原因
- 大事件發生使市場波動度增加 =>選擇權時間價值非預期上升=>CV低估
- 觀察中發現 CV model 的低估 與 S&P500 ETF 前一天收盤價與當天開盤價的價差存在關聯

Uncertain Stochastic Option Pricing in the Presence of Uncertain Jumps

J. Chirima et al. (2019)

說明不確定性以兩種形式表達：

1. 認知不確定性 (Aleatoric Uncertainty): 源於隨機實驗的不可預測性
 - a. 重要假設: 隨機實驗的大量歷史數據是可用的
 - b. ex: 擲骰子看點數、布朗運動等隨機過程
 - c. 可透過機率論建模減少此種不確定性
2. 隨機不確定性 (Epistemic Uncertainty): 由缺乏對事實完全或確切的了解驅動的。
 - a. ex: 人類可能無法得知, 市場突發事件所造成的確切影響
 - b. 無法消除

1. 研究背景(con'd)

CV model預測失準的原因猜想：

即使CV model 可在訓練期間(~5 days)盡量消除認知不確定性

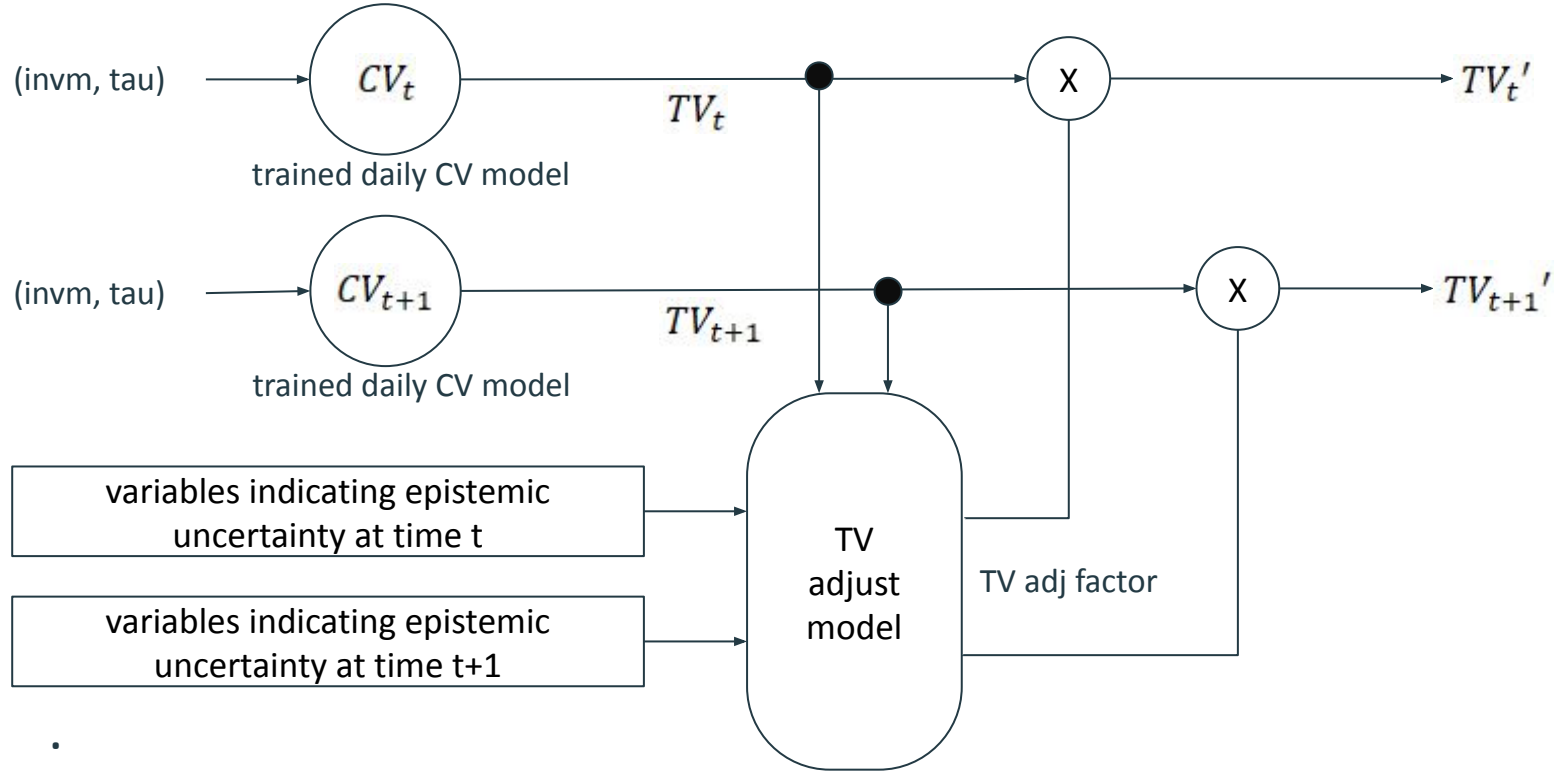
但是有關重大事件是否發生的隨機不確定性則無法學習到。

修正方法：

引入一個訓練過程完全獨立於CV model的全新模型，包含：

- 夠長的訓練期，足以涵蓋有無重大事件的日子
- 涉及隨機不確定性、會對市場重大事件做出反應的變數

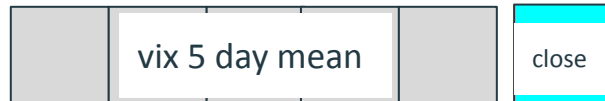
用以輔助or修正CV model的預測



1. 重大事件不只於prediction date, 出現於train date內也會影響訓練

1. sol: 簡單囊括重大事件日 + 其後5天
2. 幾乎包含200810、202003兩個月份 => 可能改觀察整個月

原標準: $\text{abs}(\text{vix}_t - \text{vix}_{5\text{day_mean}}) > \text{threshold} \ \& \ \text{vix}_t - \text{vix}_{t-1} > \text{threshold}$



原sample:

```
'''2001-09-17', '2008-09-29', '2008-10-15', '2008-10-20', '2008-10-24',  
    '2009-01-20', '2010-05-20', '2011-08-08', '2015-08-24', '2018-02-05',  
    '2020-02-27', '2020-03-09', '2020-03-12', '2020-03-16', '2020-06-11',  
    '2021-01-27', '2021-11-26''''
```

2. 資料處理-特徵選擇

欲輸入模型的特徵分為兩類

1. 用於捕捉隨機不確定性的變數

1.1. VIX隔夜變化量
$$\delta_{vix}(t) = vix_{open}(t) - vix_{close}(t - 1)$$

1.2. 標準化的 標的物隔夜報酬率
$$r_{OC}(t) = (\log S_{open}(t) - \log S_{close}(t - 1)) / vix_{close}(t - 1)$$

2. 用於辨別選擇權的種類, 使模型在面對不同種類的選擇權時間序列時, 能學習其對隨機不確定性敏感度的差別

2.1. 對數價性logm
$$logm(K, t) = \log(K / S_{close}(t))$$

2.2. time to maturity(tau)
$$\tau(t)$$

2.3. 原CV model的time value預測值TV_pred
$$TV_{pred}(K, t)$$

2. 資料處理-特徵選擇(con'd)

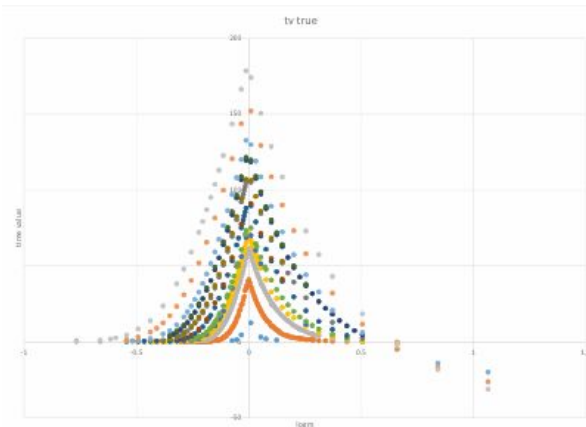
由於選擇權時間價值之於logm存在kink, 故我們仍須做額外處理如下:

$$\log m_{ITM} = \begin{cases} 0, & \text{if } \log m \geq 0 \\ -\log m, & \text{otherwise} \end{cases}, \log m_{OTM} = \begin{cases} \log m, & \text{if } \log m \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

透過增加資料維度, 將TM與OTM分為兩條相異曲線去擬合

故dataset總共6個變數, adj model的output由下式得出:

$$\text{Adj}(K, t) = \text{Model}(\delta_{vix}(t), r_{OC}(t), \log m_{ITM}(K, t), \log m_{OTM}(K, t), \tau(t), \text{TV}_{pred}(K, t))$$



2. 資料處理-序列整理

1. 時間序列的train_test_split: 非單純分割資料即可, train, val, test dataset覆蓋的時間段不能重疊
2. 模型需充分學習不同形式的重大波動

sol: 觀察重大事件發生日的密集程度, 擷取具代表性的時間段

20010917, 20090120, 20150824, ...: 突發性單日重大波動

201606: 基準月, 無重大波動

200810, 202003: 具持續性的重大波動

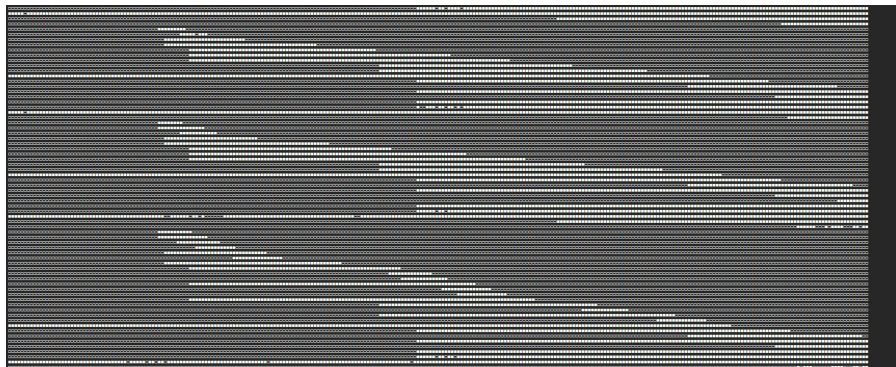
最終我們選擇2001/09、2008/10、2016/05作為train time series dataset

2020/03則做為test time series dataset

```
vix_fluct = vix[(vix['Diff'].abs() >= min_big[1]) & (vix['Normalized_Diff'].abs() >= min_big[2])]
''' '2001-09-17', '2008-09-29', '2008-10-15', '2008-10-20', '2008-10-24',
    '2009-01-20', '2010-05-20', '2011-08-08', '2015-08-24', '2018-02-05',
    '2020-02-27', '2020-03-09', '2020-03-12', '2020-03-16',
    '2020-06-11', '2021-01-27', '2021-11-26''''
```

2. 資料處理-序列整理(con'd)

- train dataset含三個時間段，每個時間段各自可整理一個時間序列矩陣，其矩陣形狀為 $(S_i, T_i, 6)$
 - S_i : 為時間序列的個數、 T_i 為時間段 i 的長度。



- 為方便洗牌訓練，我們將三個矩陣進行拼接，變成唯一一個矩陣，形狀為

$$\left(\sum_{i=1}^3 S_i, \max_{1 \leq i \leq 3} T_i, 6\right)$$

3. adj model設計

- 使用模型:GRU
 - 模型結構較簡單, 可減少訓練時間並避免過擬合
- epoch:750
 - 因RNN類模型收斂能力強, 不須過多 epoch
- 以0.994的比率遞減

3. adj model設計-預測目標

該模型將嘗試3種訓練目標，並透過實驗比較其對CV model的改善程度

將 tv_true 與 tv_pred 利用其平均值與標準差，進行標準化之後：

1. 直接預測 tv_true
2. 預測 $tv_true - tv_pred$
3. 預測 tv_true/tv_pred

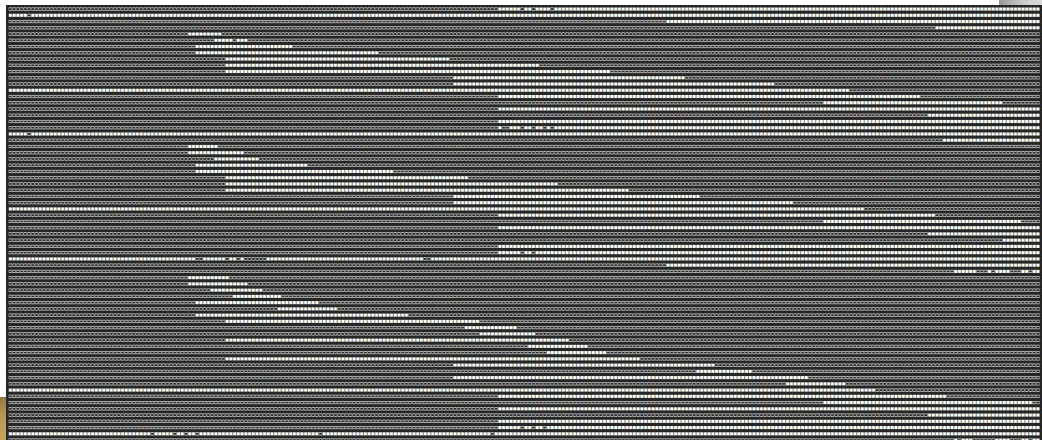
3. adj model設計-loss function

由於：

1. 整理後的time series dataset存在不含實際數據的資料點，其不應影響到模型訓練
2. 不同選擇權的出現頻率不同，應考慮重要性的異同

loss function設計：仿照CV model

- criterion : mse + mape
- weighting :
 - 計算每日、每個 logm的Kernel Density
 - 無數據的資料點權重為 0
 - 同時加權計算並達到 masking的效果

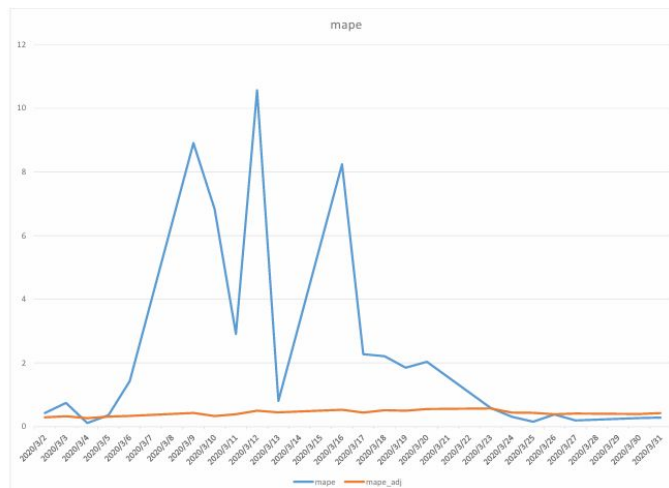
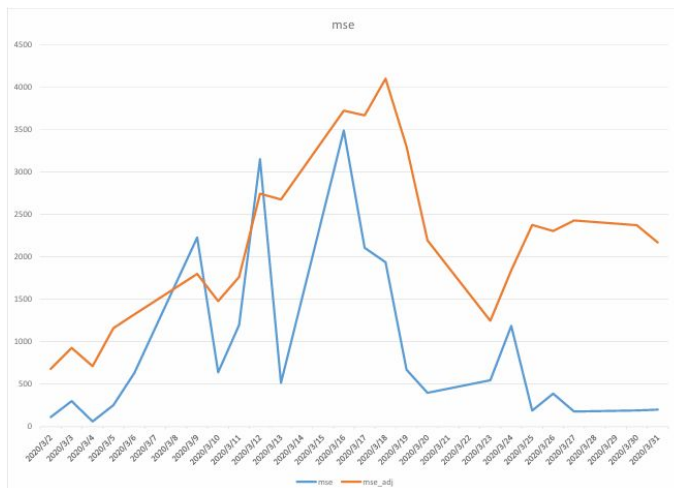


4. 實驗結果

實驗: 使用adj model直接預測tv_true

實驗結果: 2020/03 的 test time series dataset 中, 每日的mse及mape

- 其mse數值多半較原先CVmodel要高, 但是mape則多半較低。
- 標準化tv_pred的過程中, 使用到了tv_true的平均值與標準差
 - 涉及到誤用 test data 進行訓練之嫌, 故結果的可參考性存在疑慮。



5. 結論

1. 時間價值的平均值與標準差均隨著全球經濟成長，故在資料前處理中納入此考量有其必要性
 - a. 如在發生重大波動的日子，CV model低估大批選擇權，則 tv_true 和 tv_pred 標準化時的修正量便有落差，故標準化將不再適用
 - b. 應考慮一個可等量修正 $true$ 與 $pred$ 的方式，如：除以當日的 VIX 指數等等。
2. 如欲引入3.1提及的 $tv_true - tv_pred$ ，由於時間價值差值同樣受全球經濟成長影響，亦需調整。
3. 如欲引入3.1提及的 tv_true / tv_pred 時
 - a. 在鄰近0的區域，可能因分母過小，導致預測目標的 值
 - b. 且兩鄰近0的值相除，值的正負號不固定，將加劇此情況的嚴重性。
4. 由於時間價值的值域可低至-100、高至400-600，難以用微小的 ϵ 值統一正負號，
 - a. 使用SoftPlus($\log(1+ex)$): 時間價值 <0 的資料點，其特徵被嚴重淡化
 - b. 仍須尋找合理調整的方法。

2. 資料處理-特徵選擇

Index Option Prices and Stock Market Momentum Kaushik I. Amin, Joshua D. Coval (2004), 可導入特徵:

1. 股價下跌會增加call和put的vol, 然call的增加幅度大於put。
2. 當股票報酬波動性增加/下降, 投資者尋求減少/增加對stock market exposure並抬高put/call的價格。
 - a. 關聯變數: call/put indicator, vix, moneyness
3. 短期選擇權(< 60天)、deepITM call和deepOTM put($S \gg K$)的估計vol最高, 從而產生skewed的vol模式。
 - a. 關聯變數: tau, moneyness

目標引入volatility的變化量

原因: 面對重大事件, **經統計發現**, volatility在不同moneyness, tau, call/put下的變化量不同

引入volatility的變化量, 或許可捕捉以上變量面對重大事件其敏感度的不同

問題: 無法取得選擇權開盤價 > 連帶無法取得開盤volatility

或許可用標的物or VIX開盤價, 可無法捕捉volatility smile

volatility smile或許可用SSVI or ds model補足, 然而這樣會失去真實資料的意義

sol: 直接引入moneyness, tau, call/put作為input?

option time series

1. adjust model的訓練流程獨立於CV model之外
2. 交易量為0時, 也會有價格變動(造市商、證交所等因素)

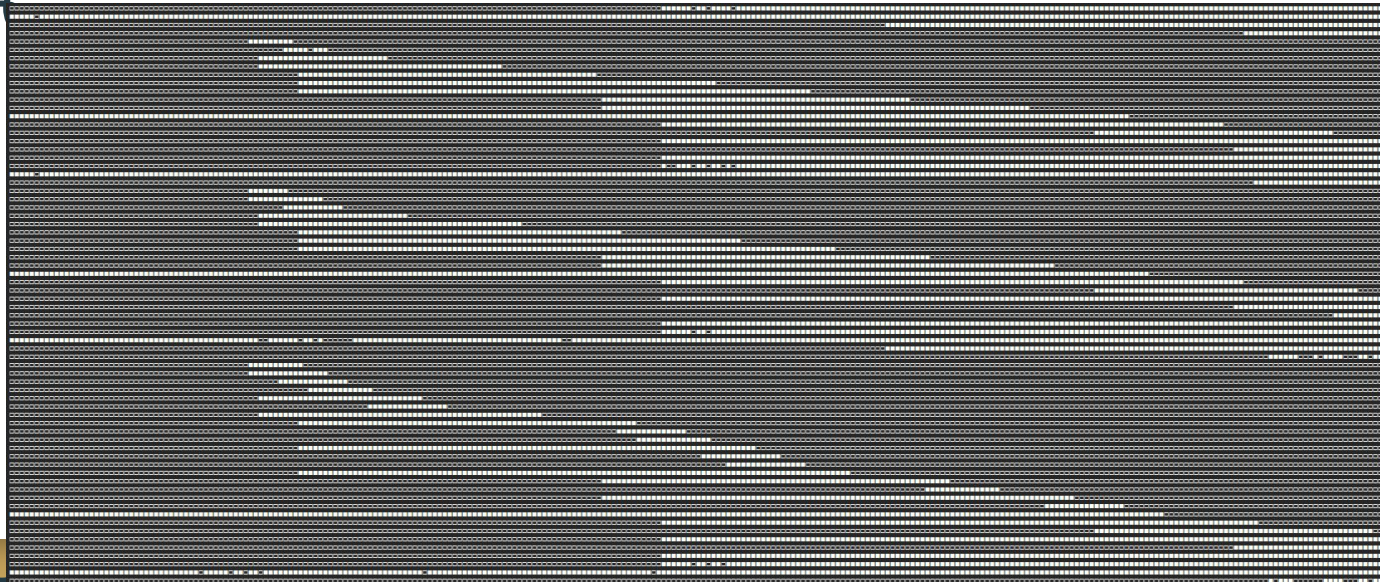
=>採用prs_dataset_involve0volume

每筆選擇權, 自issue到expire, 為長度不等的time series(附圖: part4內所有種類選擇權的時間序列).

Sol: mask

date(from 20200103-20210205)

different
kinds of
option



多維度時間序列input範例：

一條序列

tensor([[[[100.0000, 0.5000, 110.0000], [101.5000, 1.0000, 30.0000]],
[[98.0000, 1.0000, 100.0000], [99.2000, 1.5000, 25.0000]],
[[95.0000, 2.0000, 105.0000], [96.7000, 2.5000, 20.0000]]]])

一種屬性 一個時間點

- 用以處理長度不等的時間序列
- padding: 將不同序列間難以對齊的部分補0, masking: 讓model在訓練過程中忽略值=0的部分

```
import numpy as np
from keras.preprocessing.sequence import pad_sequences
```

```
sequences = [
    [1, 2, 3],
    [2, 3, 4, 5, 6],
    [3, 4]
]
```

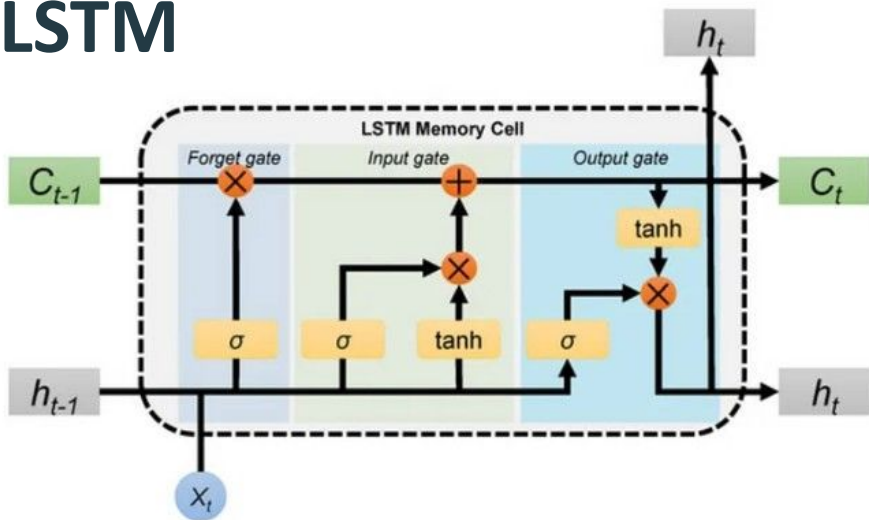
```
padded_sequences = pad_sequences(sequences, padding='post')
print(padded_sequences) # [[1 2 3 0 0] [2 3 4 5 6] [3 4 0 0 0]]
```

```
from tensorflow.python.keras.models import Sequential
from tensorflow.python.keras.layers import Masking, LSTM, Dense
```

建立模型, 使用 Masking 忽略值 0

```
model = Sequential([
    Masking(mask_value=0.0, input_shape=(None, 1)), # 忽略填充值 0
    LSTM(32, return_sequences=False), # 使用 LSTM
    Dense(1) # 預測輸出層
])
```

LSTM



c_t : output at time t

h_t : hidden state at time t

x_t : input at time t

門閥特點：

1. 可決定保留多少過去資訊
2. 可決定引進多少現在資訊
3. 可決定傳遞多少未來資訊

forget gate(0~1): 決定忘記多少 c_{t-1}

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

input gate(0~1): 決定引進多少比例的 c_t

$\text{input}(h_{t-1}, x_t)$ 經 $\tanh$ 變成一個向量 $\tilde{c}_t$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

output gate(0~1): c_t 經 $\tanh$ 形成的向量

透過 output gate 決定儲存多少在 h_t 內

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

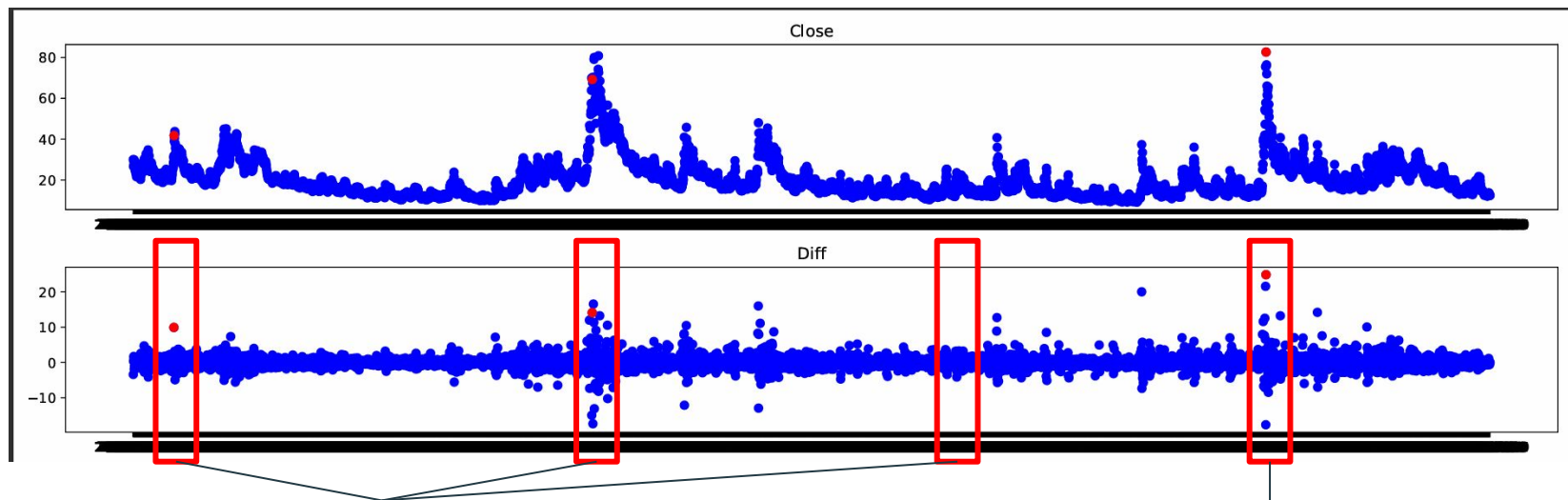
$$h_t = o_t * \tanh(C_t)$$

使用既有模型做prediction的程序

1. 決定最早要取到何時的時間序列決定start_date)
2. 基於欲做預測的資料: 依照(strike_price, tau)確定含有的選擇權種類
3. 向前獲取資料, 得到 $t \geq \text{start_date}$ 的指定選擇權資料
4. 將資料重新整理成時間序列形式
5. 丟入模型中做預測

時間序列節錄

```
vix['5Day_Mean'] = vix['Close'].shift(1).rolling(window=5).mean()  
vix['5Day_STD'] = vix['Close'].shift(1).rolling(window=5).std()  
vix['Diff'] = vix['Close'].diff()# / vix['5Day_STD']  
vix['Normalized_Diff'] = (vix['Close'] - vix['5Day_Mean'])# / vix['5Day_STD']
```



train dataset

test dataset

模型訓練實驗

input feature: logm, tau, s&p500ETF隔夜價差(from竣瑜學長), vix隔夜差

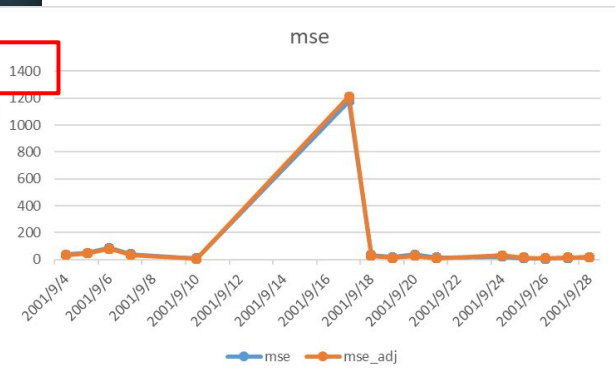
target: $(tv_true - tv_pred) / vix$, loss_function: mse

觀察結果: mape顯著降低, 但mse改善程度有限

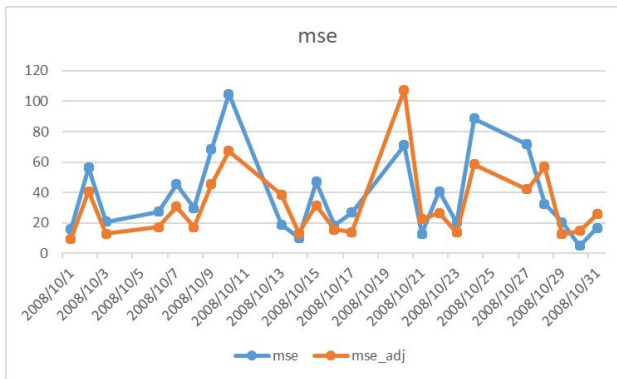
且當市場震動劇烈(原始mse破千)時, 改善甚微

train dataset 2001/09 2008/10 2016/05

2001/09



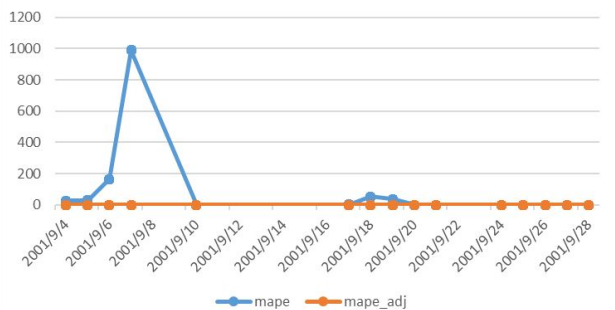
2008/10



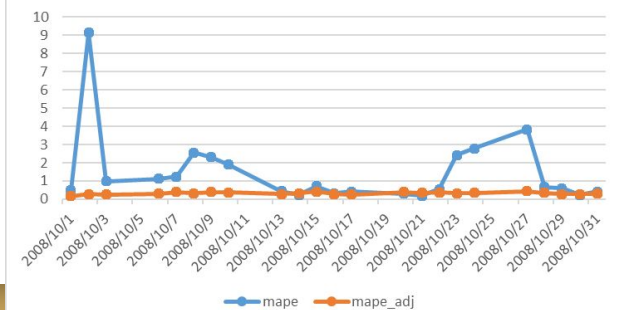
2016/05



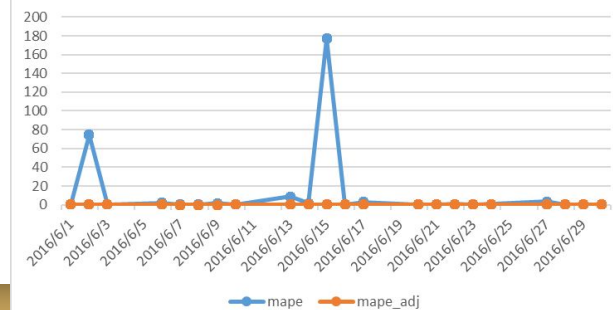
mape



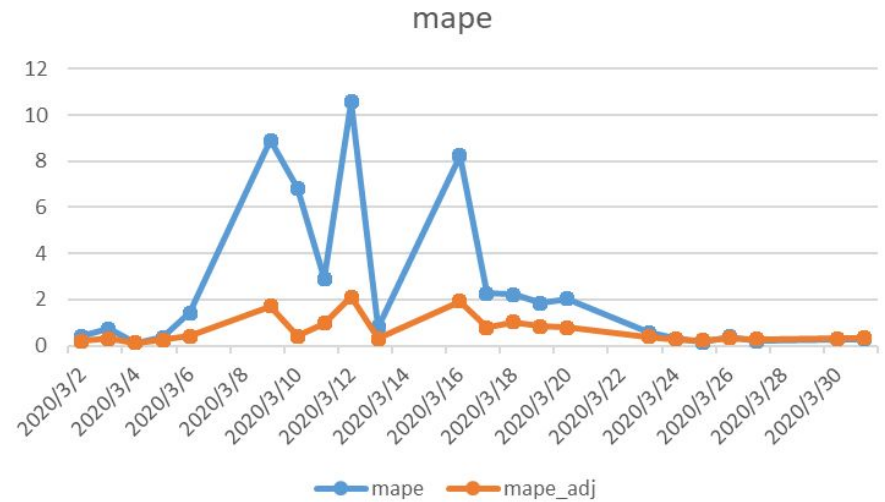
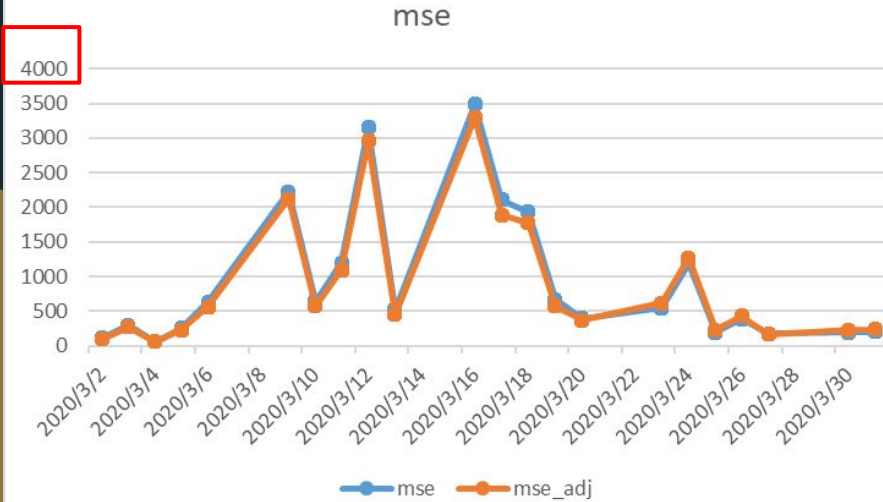
mape



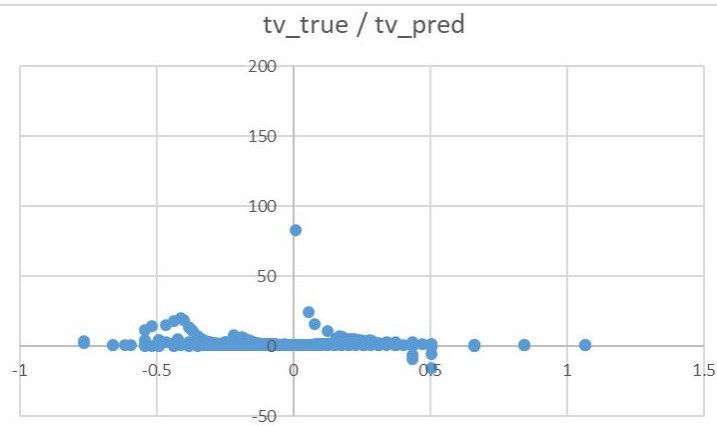
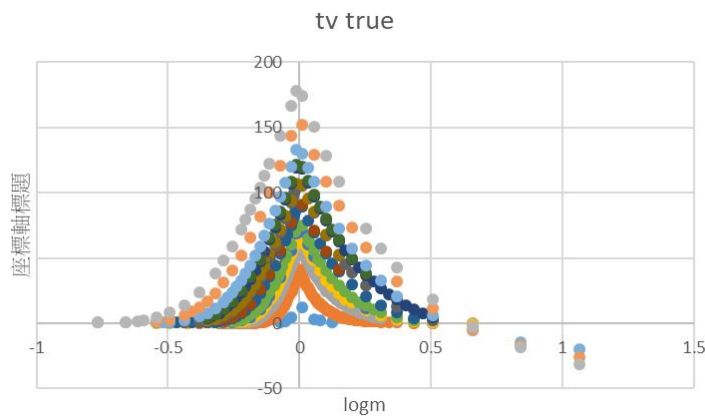
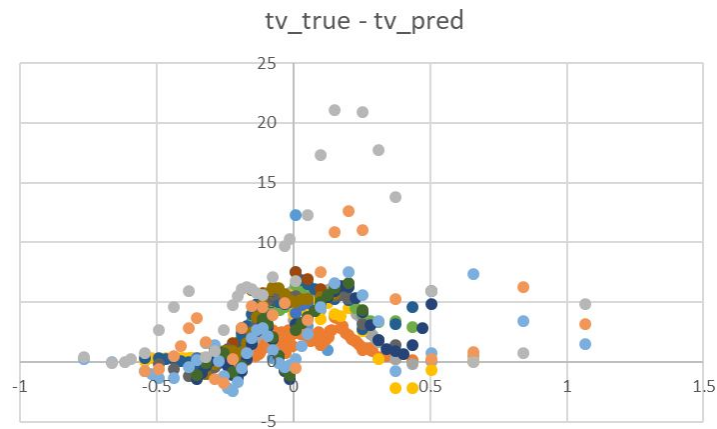
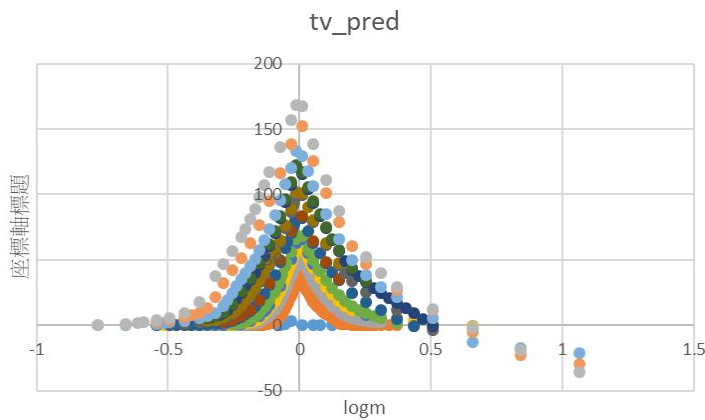
mape



test dataset 2020/03



test dataset內 某日的tv_pred相對tv_true (x軸: logm)



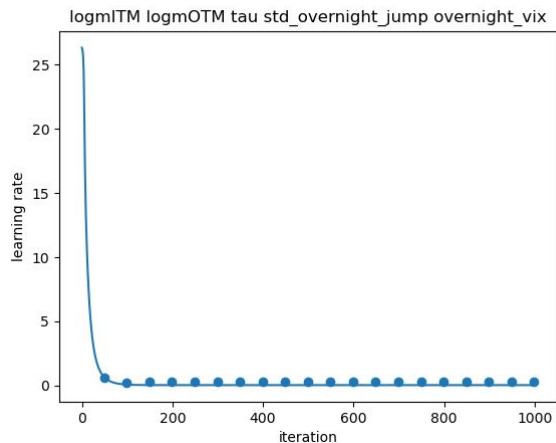
1. pred和true的diff(pred - true)過於雜亂
 - a. 可能是造成test loss難以降低的原因
2. pred和true的比例(pred / true)較為整齊, 但易有極大值
 - a. 如能處理極大值, 或許是較好的target

target: $(tv_true - tv_pred) / vix$

loss_function: mse

train loss和test loss有差距

模型泛化能力有限



```
49/1000, Epoch 50, Train loss: 0.6760, Test loss: 0.5940, C
99/1000, Epoch 100, Train loss: 0.0789, Test loss: 0.2591, C
149/1000, Epoch 150, Train loss: 0.0391, Test loss: 0.2864, C
199/1000, Epoch 200, Train loss: 0.0361, Test loss: 0.3034, C
249/1000, Epoch 250, Train loss: 0.0362, Test loss: 0.3071, C
299/1000, Epoch 300, Train loss: 0.0361, Test loss: 0.3077, C
349/1000, Epoch 350, Train loss: 0.0360, Test loss: 0.3077, C
399/1000, Epoch 400, Train loss: 0.0361, Test loss: 0.3077, C
449/1000, Epoch 450, Train loss: 0.0361, Test loss: 0.3080, C
499/1000, Epoch 500, Train loss: 0.0362, Test loss: 0.3076, C
549/1000, Epoch 550, Train loss: 0.0359, Test loss: 0.3078, C
599/1000, Epoch 600, Train loss: 0.0364, Test loss: 0.3078, C
649/1000, Epoch 650, Train loss: 0.0361, Test loss: 0.3081, C
699/1000, Epoch 700, Train loss: 0.0363, Test loss: 0.3084, C
749/1000, Epoch 750, Train loss: 0.0362, Test loss: 0.3080, C
799/1000, Epoch 800, Train loss: 0.0359, Test loss: 0.3076, C
849/1000, Epoch 850, Train loss: 0.0358, Test loss: 0.3082, C
899/1000, Epoch 900, Train loss: 0.0358, Test loss: 0.3081, C
949/1000, Epoch 950, Train loss: 0.0361, Test loss: 0.3078, C
999/1000, Epoch 1000, Train loss: 0.0359, Test loss: 0.3081, C
```

VIX rolling variance(window=5days)

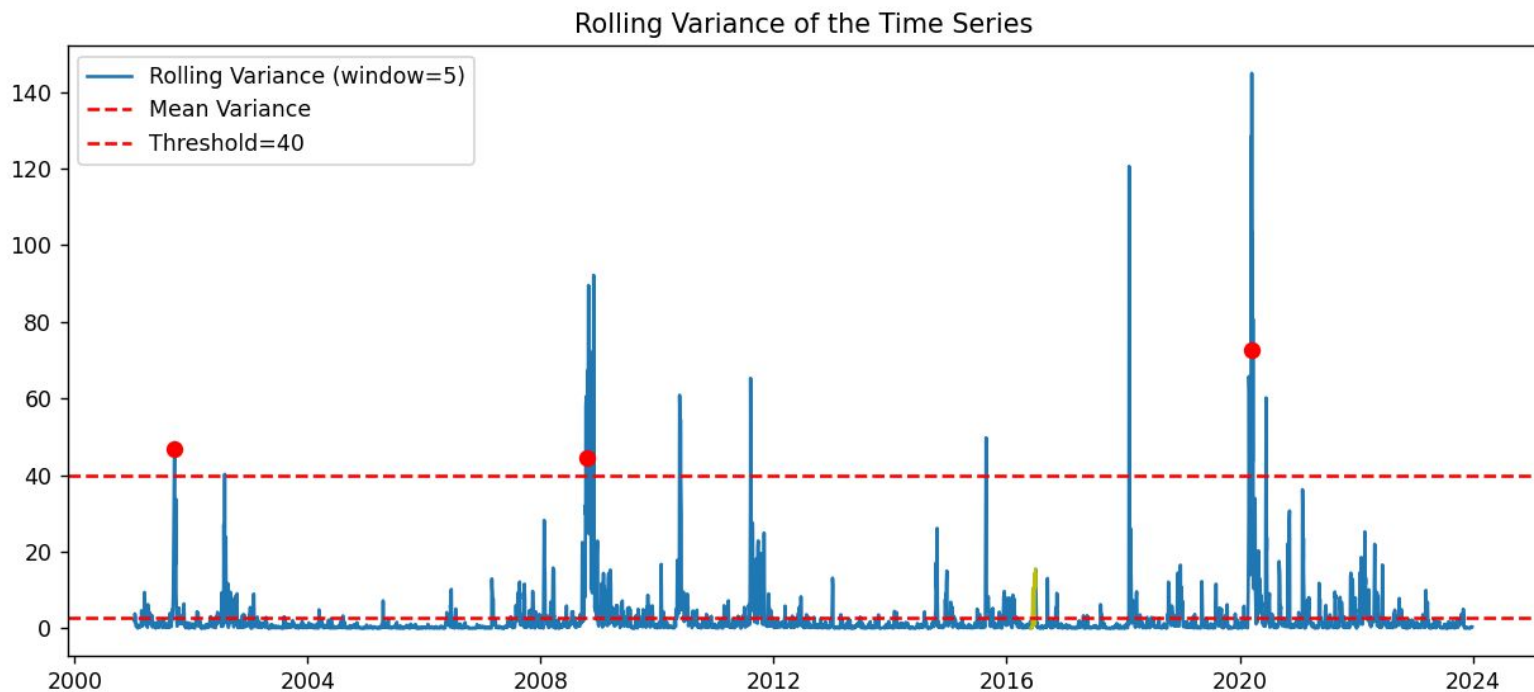
藍線:vix指數的
rolling variance

紅點:大事件日
(from 竣瑜學長論文)

黃線:基準月
(from 竣瑜學長論文)

先抓出有大var的點

證實有低估



1. 重大事件發生於training dates



2. 重大事件發生於test date



先前模型結果

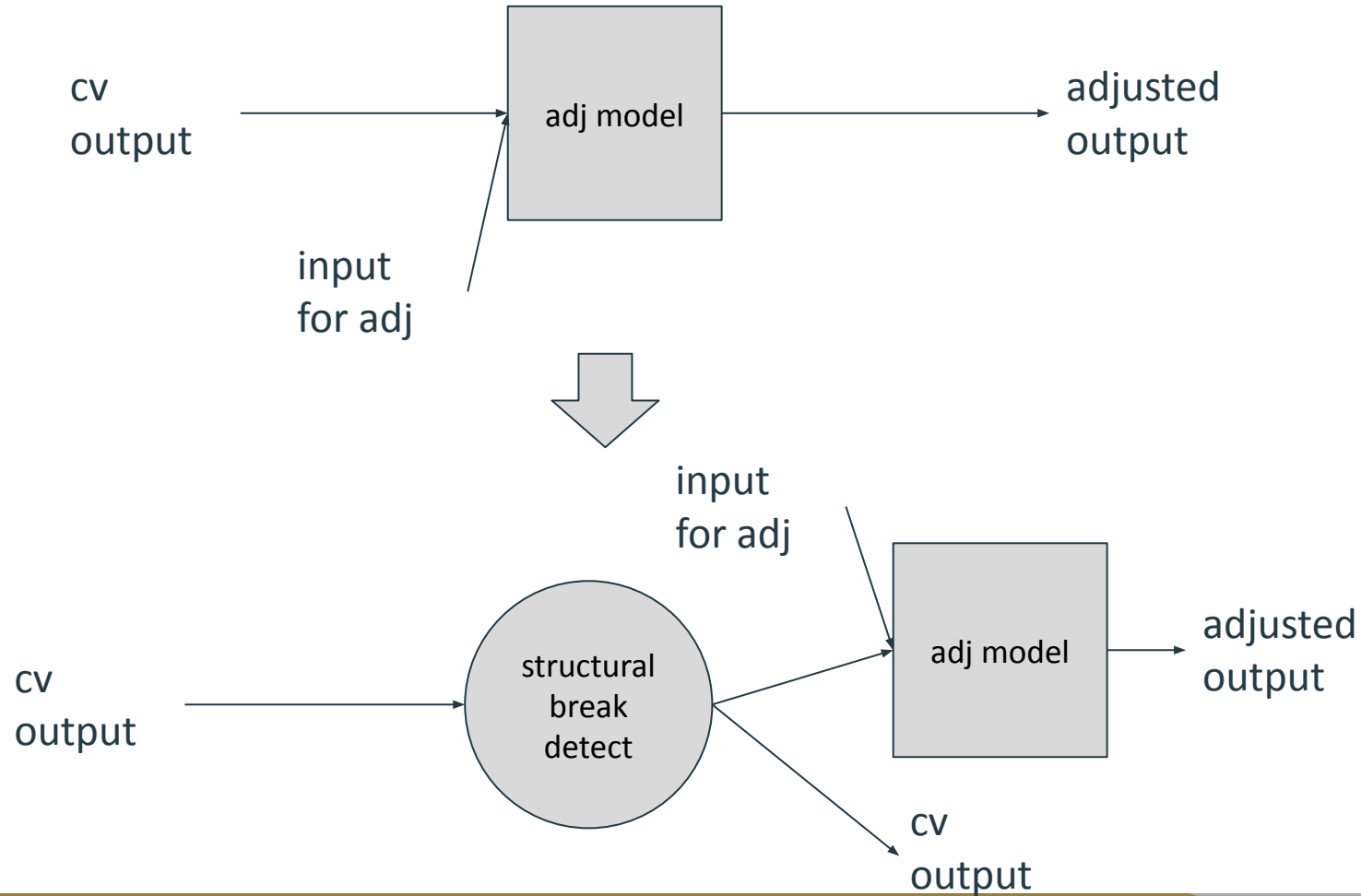
對於預測錯誤的調整不足，僅在mape有所改善、mse減少微乎其微

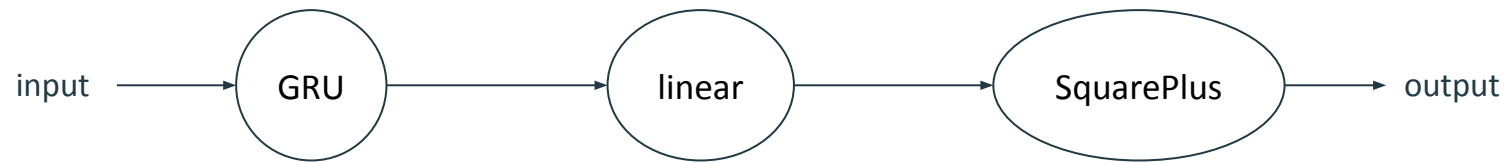
推測原因：輸入的時間序列資料過多、使模型難以適應所有情況，解決方法：

一律調整cv model output => 偵測是否有structural break, 有才調整

優點：

1. 所需的資料量較整條時間序列要少
2. 模型面臨的資訊複雜度降低，





oversampling